gpod

GNSS Satellite Precise Orbit Determination (POD)
Extended Kalman Filter + high-fidelity force model + numerical integration.
Supports **GPS / BDS / GLONASS / Galileo** with **centimeter-level** orbit accuracy.

`gpod` is a pure-C (C89/C99) orbit determination library. Its public interface is a single header, `src/podlib.h`. The satellite state is propagated in the ECI frame with a complete force model, and an EKF ingests ionosphere-free GNSS observations from a ground station network to estimate orbits, SRP parameters and clocks.

Contents

- **Getting started**
  - 1. Overview
  - 2. Feature Summary
  - 3. Quick Start
- **Algorithm**
  - 4. Algorithm Framework
    - Satellite State and Dynamics
    - Force Model Details
    - State Transition Matrix
    - Observation Model
    - Filtering: Time and Measurement Updates
    - Processing Strategy
    - Mapping to `gpod.pdf`
    - Quality Control: Cycle Slips and Outliers
  - 5. Repository Layout
- **Code reference**
  - 6. Code Structure
    - Module Responsibilities
    - Module API Index
    - Module Architecture and Dependencies
    - End-to-End Data Flow
    - Naming and Coding Conventions
    - Extension Guide
  - 7. Core Data Structures
    - Top-Level Container
    - Filter, Solution and Observation Containers
    - Estimated State Vector

        - Force Model Structure
        - Satellite Metadata Table `gsatinfos[256]`
    - 8. Key Workflows and Call Chains
        - Initialization
        - Per-Epoch Filtering
        - The Two Estimation Modes
        - Propagator and Transition Matrix
    - 9. Configuration Reference
        - Force Model Options
        - Processing Options
        - Integrator Selection
        - Compile-Time Switches
        - State Process Noise in the Example
- **Running gpod**
    - 10. Data Preparation
        - Required Files
        - File Format / Reader Map
        - Memory Footprint and Sizing
        - Test Dataset
    - 11. Build
        - Requirements
        - Build Steps
        - Linking From Your Own Project
        - Troubleshooting
    - 12. Running and Output
    - 13. Validation
    - 14. Known Limitations
- **Appendices**
    - 15. Glossary
    - 16. References
    - 17. Contact and Licensing

---

1. Overview

`gpod` couples two things that are usually kept apart:

Concern	What `gpod` does
Dynamics	Propagates the satellite state in the **ECI (inertial) frame** under a complete force model (gravity field, third body, solar radiation pressure, drag, relativity, tides) using one of nine numerical integrators
Estimation	Ingests **ionosphere-free pseudorange/phase** observations from a ground network and runs an EKF that estimates satellite orbits, SRP/empirical force parameters, satellite clocks, receiver coordinates, receiver clocks, tropospheric delay and ambiguities
Covariance	Builds the state transition matrix either by integrating the **variational equations** (default) or by **numerical differentiation**, keeping long-arc filtering numerically stable

Design principles visible throughout the code:

- **One header per layer** — `rtklib.h` (platform/utility types) and `podlib.h` (POD API); every other implementation detail is `static`.
- **Configuration by structs, not globals** — `force_model_opt_t` and `pod_opt_t` carry every switch, model order, file path and process-noise value; `podinit()` deep-copies the options.
- **Model data sharing** — in a multi-satellite run one copy of the EOP, space-weather, gravity and ocean-tide models is shared across all satellites (`usesamegravitymdl`, `usesameeop`, `usesamespw`, `usesameotm`).
- **Two operating modes** — orbit fitting from given satellite positions (mode A) and multi-station joint estimation from GNSS observations (mode B), dispatched by a single `podflt()` entry point.
- **Vendored dependencies** — SOFA, NRLMSISE-00, OpenBLAS, GSL and Boost.Odeint build from source
  with the library, so there is no external package hunt.

---

2. Feature Summary

Category	Capability
Constellations	GPS, BDS, Galileo, GLONASS (framework also keeps QZSS / IRNSS / SBAS / LEO interfaces); selection is **compile-time**, see 9.4
Estimation	Extended Kalman Filter with forward, backward, forward/backward combination (`podfbcmb`, `podfbcmbe`) and RTS smoothing (`podsatorbfit` type 1)
Gravity field	Spherical-harmonic expansion, normalized coefficients read from `.gfc` (GGM03C / EGM2008 selectable), arbitrary order, tide system selectable
Tides	Solid earth tide (IERS 2010, optional `dehanttideinel`), permanent tide, rotational deformation, anelastic earth, pole tide, ocean tide (analytic default or external ocean-tide model file)
Third body	JPL **DE440** planetary ephemeris with Chebyshev interpolation — Sun, Moon and eight planets

Category	Capability
Solar radiation pressure	7 models: Default, ECOM1(5), ECOM1(9), ECOM2, ECOMC, CODE(5), CODE(9), with 13 estimable parameters in the D/Y/B satellite frame
Shadow	Cylindrical, geometrical and conical shadow/penumbra models
Atmospheric drag	Harris-Priester or **NRLMSISE-00** density, relative wind accounting for Earth rotation and LOD
Relativity	First-order post-Newtonian acceleration with both gravitational and velocity-term partials
Numerical integration	9 methods across GSL `odeiv2`, Boost.Odeint and a built-in RKF7(8); the variational equations are integrated **inside** the same ODE system
Observation model	Ionosphere-free (IF) pseudorange + phase, satellite and receiver PCV, phase wind-up, Earth-rotation-corrected receiver position, tidal station displacement, DCB, elevation cutoff
Quality control	LL / GF / MW cycle-slip detection, MAD outlier detection and satellite exclusion, eclipse-exclusion for block IIA
Data interfaces	RINEX 2/3 observation/navigation/clock files, SP3 precise ephemeris, DCB, SINEX, ATX, BLQ, GFC, DE440, IERS EOP, space weather
Timescales	UTC / GPST / BDT / TT / TDB / UT1 / MJD with leap seconds and EOP-driven UT1-UTC
Logging	Leveled trace (`1` error … `5` verbose matrix dumps), compile-time removal via `OUTPUT_LOG`

---

3. Quick Start

```
# 1. build (bundled dependencies are built from source)
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . -j8

# 2. run from a directory that contains data/ (see section 10)
./test_podflt
```

Minimal integration into your own program — this mirrors `test_podflt.c`:

```
#include "src/podlib.h"

pod_t     pod = {0};
pod_opt_t opt = {0};

/* 1. force model: which forces, which models, which files */
force_model_opt_t fm = {0};
fm.gravity    = 1;                    fm.nbody   = 1;   fm.solrad = 1;
```

```
fm.atmdrag    = 1;                    fm.atmdensopt = ATMODENSITY_NRL;
fm.solidtide  = 1;                    fm.solradmdl  = SOLRAD_MODEL_ECOMC;
fm.shmopt     = SHADOW_CYLINDRICAL;   fm.gN = 10;  fm.gM = 10;  fm.oL = 10; fm.oM = 10;
fm.udtransmatrix = UDTRANSMATRIX_INTGRTN;   /* variational equations */
fm.udacclgrad    = ACCLGRAD_ANALYSIS;       /* analytic acceleration partials */
fm.usesamegravitymdl = fm.usesameeop = fm.usesamespw = fm.usesameotm = 1;
strcpy(fm.eopfile,      "data\\model\\EOP-All.txt");
strcpy(fm.spwfile,      "data\\model\\SW-All.txt");
strcpy(fm.gfcfile,      "data\\model\\GGM03C.gfc");
strcpy(fm.jplde440file, "data\\model\\de440coeff");
strcpy(fm.octfile,      "data\\model\\fes2004_Cnm-Snm.dat");
for (int i = 0; i < MAXSAT; i++) opt.fmdlopts[i] = fm;

/* 2. estimation options: process noise, files, a priori mode */
opt.estopt        = prcopt_default_ppp;
opt.estopt.nf     = 2;                /* ionosphere-free combination */
opt.estopt.elmin  = 10.0*D2R;
opt.estopt.navsys = SYS_GPS;
opt.estopt.posopt[0] = opt.estopt.posopt[2] = opt.estopt.posopt[3] = 1;
opt.udorbitint    = ORBITINT_ODEINTV2_RKF78;
opt.estmode       = GPOD_FIXRCVPOS_ESTSATPOS;   /* mode A or mode B */
strcpy(opt.navfile, "data\\nav\\BRDC00IGS_R_20201880000_01D_MN.rnx");
strcpy(opt.clkref,  "xxxx");          /* reference clock station */

/* 3. initialize, load a priori ephemeris, then filter epoch by epoch */
podinit(&pod, &opt);                                  /* models + station table + ATX/DCB */
readrnx(opt.navfile, 0, "", NULL, &pod.nav, NULL);    /* broadcast ephemeris */
readsp3("data\\nav\\igs21130.sp3", &pod.nav, 0);       /* precise ephemeris (reference) */

podobs_t obs;
while (fread(&obs, sizeof(obs), 1, fp_obs)) {         /* binary observation packets */
    podflt(&pod, &obs);                               /* one EKF epoch: x, P -> pod.sol */
}

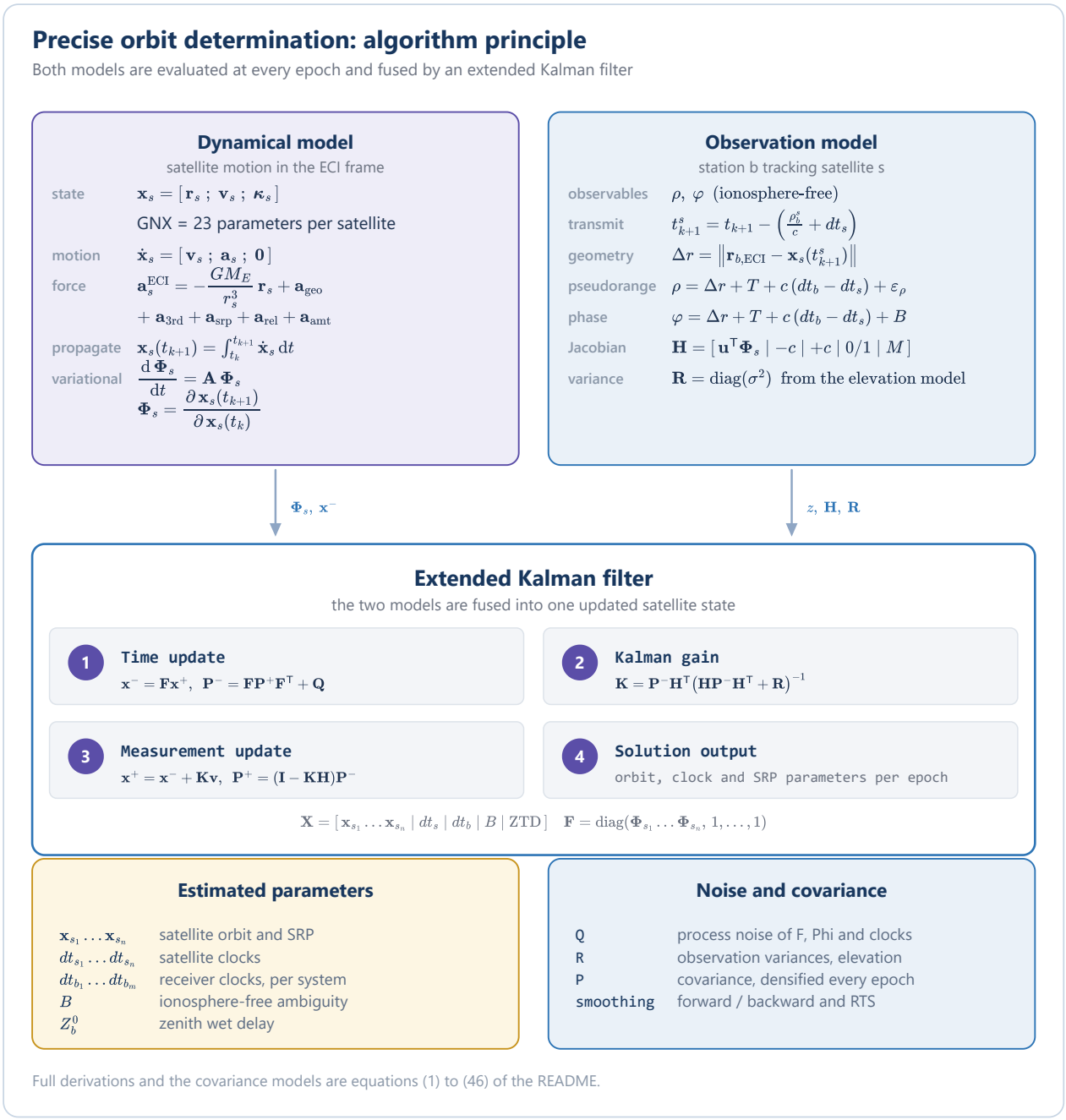
podfree(&pod);                                        /* release every model and matrix */
```

`podflt()` is the only call inside the loop: it dispatches on `obs.type`, performs the time update (force-model propagation + transition matrix), builds the measurement Jacobian and runs the EKF measurement update. The four calls you must not forget are `podinit()` → a priori ephemeris → the `podflt()` loop → `podfree()`.

---

4. Algorithm Framework

Sections 4.1 to 4.5 follow the derivation in `gpod.pdf`; equations (5) to (13), (18) to (19), (23), (26), (33), (35), (37) to (38) and (41) to (46) are added here because the implementation relies on them. The mapping between the two numbering schemes is given in 4.7.



The method has three parts that are evaluated together at every epoch. The **dynamical model** propagates the satellite state in the ECI frame with a full force model and delivers the state transition matrix. The **observation model** turns the station measurements into modelled observables, their design matrix and their variances. The **extended Kalman filter** fuses the two into an updated state, which then becomes the initial state of the next epoch. The subsections below derive each part; section 8.2 maps them onto the call chain.

4.1 Satellite State and Dynamics

The estimated state of satellite *s* is

$$\mathbf{x}_s = \begin{bmatrix} \mathbf{r}_s^{\mathrm{ECI}} \\ \mathbf{v}_s^{\mathrm{ECI}} \\ \boldsymbol{\kappa}_s \end{bmatrix} \tag{1}$$

where $\mathbf{r}_s^{\mathrm{ECI}}$ and $\mathbf{v}_s^{\mathrm{ECI}}$ are the ECI position and velocity and $\boldsymbol{\kappa}_s$ collects the orbit-dynamics model parameters (SRP and empirical force parameters). The differential equation is

$$\dot{\mathbf{x}}_s = \begin{bmatrix} \dot{\mathbf{r}}_s^{\mathrm{ECI}} \\ \dot{\mathbf{v}}_s^{\mathrm{ECI}} \\ \dot{\boldsymbol{\kappa}}_s \end{bmatrix} = \begin{bmatrix} \mathbf{v}_s^{\mathrm{ECI}} \\ \mathbf{a}_s^{\mathrm{ECI}} \\ \mathbf{0} \end{bmatrix} \tag{2}$$

$$\mathbf{a}_s^{\mathrm{ECI}} = -\frac{GM_E}{r_s^3}\mathbf{r}_s + \mathbf{a}_{geo} + \mathbf{a}_{3rdbody} + \mathbf{a}_{srp} + \mathbf{a}_{rel} + \mathbf{a}_{amt} + \boldsymbol{\epsilon} \tag{3}$$

Term	Meaning	Implementation
$-GM_E\,\mathbf{r}_s/r_s^3$	Two-body central attraction	`gravityforce()`
$\mathbf{a}_{geo}$	Gravity field correction: spherical harmonics + solid/ocean/pole tide + rotational deformation	`gravityforce()`, `solidtidecorr()`, `oceantidecorr_*()`, `poletidecorr()`, `rotcorr()`
$\mathbf{a}_{3rdbody}$	Third-body point-mass attraction	`nbpertnforce()`, `pointmassforce()`
$\mathbf{a}_{srp}$	Solar radiation pressure	`solradpforce()`
$\mathbf{a}_{rel}$	Relativistic correction	`relattyforce()`
$\mathbf{a}_{amt}$	Atmospheric drag	`atmdragforce()`
$\boldsymbol{\epsilon}$	Unmodeled acceleration	absorbed by the estimable parameters $\boldsymbol{\kappa}$

The five components are evaluated independently and summed by `satforce()`:

$$\mathbf{a}_{total} = \mathbf{a}_{gm} + \mathbf{a}_{srp} + \mathbf{a}_{ad} + \mathbf{a}_{np} + \mathbf{a}_{rel}$$

State propagation is numerical:

$$\mathbf{x}_s(t_{k+1}) = \int_{t_k}^{t_{k+1}} \dot{\mathbf{x}}_s\, dt \tag{4}$$

4.2 Force Model Details

Gravity field. The geopotential is expanded in normalized spherical harmonics,

$$V(r,\phi,\lambda) = \frac{GM}{r}\sum_{n=0}^{N}\left(\frac{R_E}{r}\right)^n \sum_{m=0}^{n} \bar{P}_{nm}(\sin\phi) \times \left[\bar{C}_{nm}\cos(m\lambda) + \bar{S}_{nm}\sin(m\lambda)\right] \tag{5}$$

with N, M set by `opt.gN` / `opt.gM` (10×10 in the example), the coefficients read from the `.gfc` file by `readgfc()` and the associated Legendre functions computed by `legendre()`. The gravitational acceleration is the gradient of the potential including the tidal contributions,

$$\mathbf{a}_{geo} = \nabla\left(V + V_{\mathrm{tide}}\right) \tag{6}$$

The permanent tide, the time-variable part of the tide potential and the deformation of the Earth are all folded into the coefficients before the gradient is taken, so that every correction is evaluated at the current satellite position:

Correction	Function	Controlled by
Solid earth tide (degree-2 and degree-3)	`solidtidecorr()`	`solidtide`, `anelasticearth`, `tidesystem`
Ocean tide	`oceantidecorr_def()` (analytic) or `oceantidecorr_otm()` (model file)	`oceantide`, `octfile`
Pole tide	`poletidecorr()`	`poletide`
Rotational deformation	`rotcorr()`	`rotcorr`

The degree-2 solid earth tide follows the IERS formulation, in which the Love number k_2 scales the external potential of the Sun and the Moon,

$$\Delta\bar{C}_{2m} - \mathrm{i}\,\Delta\bar{S}_{2m} = \frac{k_2}{\sqrt{5}} \sum_{p\in\{sun,\,moon\}} \frac{GM_p}{GM_E}\left(\frac{R_E}{r_p}\right)^3 \times \bar{P}_{2m}(\sin\phi_p)\, e^{\,\mathrm{i}m\lambda_p} \tag{7}$$

with the degree-3 term and the permanent tide added on top; `anelasticearth` switches to the anelastic Love numbers.

Third body. For each perturbing body p (Sun, Moon and the eight planets from the DE440 cache),

$$\mathbf{a}_{3rdbody} = \sum_p GM_p \left(\frac{\mathbf{r}_p - \mathbf{r}}{\|\mathbf{r}_p - \mathbf{r}\|^3} - \frac{\mathbf{r}_p}{\|\mathbf{r}_p\|^3} \right) \tag{8}$$

Solar radiation pressure. In the satellite body frame defined by the Sun direction **D**, the solar-panel axis **Y** and **B = D × Y**, the acceleration is

$$\mathbf{a}_{srp} = \nu\, C_R \frac{A}{m} P_0 \left(\frac{A_u}{\|\mathbf{r}_{sun} - \mathbf{r}_s\|} \right)^2 \hat{\mathbf{u}} \times (1 + \text{model terms}) \tag{9}$$

where ν is the shadow factor returned by the selected shadow model,

$$\nu = \begin{cases} 0 & \text{full shadow} \\ f(\text{geometry}) \in (0,1) & \text{penumbra} \\ 1 & \text{full sunlight} \end{cases} \tag{10}$$

and is computed by `shadowcylindrical()`, `shadowgeometrical()` or `shadowconical()`. C_R is the reflectivity, A the cross-section, m the mass (`gsatinfos[].mass`), P_0 the solar radiation pressure at 1 AU and A_u the astronomical unit. The example uses `P0 = 4.55982118135874e-06 N/m²` (= 1367 W/m² / c) and `Au = 149597870700 m`. The estimable parameters, all initialized to `1E-30` (effectively fixed) and relaxed by `satvar[]` / `satprn[]`, are `D0, Dc, Ds, Dc2, Ds2, Dc4, Ds4, Y0, Yc, Ys, B0, Bc, Bs`.

`solradmdl`	Parameter set used
`SOLRAD_MODEL_DEFAULT`	D0, Y0, B0, Bc, Bs (+ opt. `CR`)
`SOLRAD_MODEL_ECOM1_5` / `ECOM1_9`	ECOM1 first-order (5/9 parameters)

`solradmdl`	Parameter set used
`SOLRAD_MODEL_ECOM2`	ECOM2 (D/2D + B terms)
`SOLRAD_MODEL_ECOMC`	ECOM+C (D, Y, B, plus 2D/4D harmonics)
`SOLRAD_MODEL_CODE_5` / `CODE_9`	CODE empirical model (5/9 parameters)

Atmospheric drag. The relative wind removes the co-rotation term, including the LOD correction to the Earth rotation rate,

$$\mathbf{v}_{rel} = \mathbf{v} - \boldsymbol{\omega} \times \mathbf{r}, \qquad \omega_z = \Omega_e - 0.843994809 \times 10^{-9}\,\mathrm{LOD} \tag{11}$$

$$\mathbf{a}_{adt} = -\tfrac{1}{2} C_D \frac{A}{m} \rho \,\|\mathbf{v}_{rel}\|\, \mathbf{v}_{rel} \tag{12}$$

ρ comes from Harris-Priester (`atmdenshpm()`, evaluated in the pseudo-inertial frame) or NRLMSISE-00 (`atmdensnrl()`, evaluated in ECEF), driven by the space-weather file read by `readspw()`. `Ω_e = 7.292115146706388e-05 rad/s`.

Relativity. The first-order post-Newtonian (Schwarzschild) term of the Earth's field,

$$\mathbf{a}_{rel} = \frac{GM_E}{c^2 r^3}\left[\left(4\frac{GM_E}{r} - v^2\right)\mathbf{r} + 4\,(\mathbf{r}\cdot\mathbf{v})\,\mathbf{v}\right] \tag{13}$$

is implemented by `relattyforce()`, together with its partials with respect to position and velocity (`reldadp()` and `reldadv()`).

4.3 State Transition Matrix

$$\boldsymbol{\Phi}_s(t_k,\, t_{k+1}) = \frac{\partial \mathbf{x}_s(t_{k+1})}{\partial \mathbf{x}_s(t_k)} \tag{14}$$

Two strategies (`force_model_opt_t::udtransmatrix`):

Strategy	Value	Mechanism	Where
Variational equations (default)	`UDTRANSMATRIX_INTGRTN`	The 23×23 matrix $\boldsymbol{\Phi}$ is appended to the ODE system, raising the integrated state from 6 to $6 + 23^2 = 535$ elements; $\dot{\boldsymbol{\Phi}} = \mathbf{A}\boldsymbol{\Phi}$	`orbitdeq()`
Two-point numerical differential	`UDTRANSMATRIX_NUMDIF_2P`	Re-propagates the arc once per state component with a $+h$ perturbation	`satorbit()`
Three-point numerical differential	`UDTRANSMATRIX_NUMDIF_3P`	Central difference with $\pm h$ perturbations	`satorbit()`

$$\frac{d\boldsymbol{\Phi}_s(t_k,t)}{dt} = \mathbf{A}\,\boldsymbol{\Phi}_s(t_k,t) \tag{15}$$

with the design matrix, in which every partial is evaluated at epoch t in the ECI frame,

$$\mathbf{A} = \begin{bmatrix} \mathbf{0} & \mathbf{I} & \mathbf{0} \\ \partial\mathbf{a}/\partial\mathbf{r} & \partial\mathbf{a}/\partial\mathbf{v} & \partial\mathbf{a}/\partial\boldsymbol{\kappa} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} \end{bmatrix} \tag{16}$$

$$\boldsymbol{\Phi}_s(t_k,\, t_{k+1}) = \int_{t_k}^{t_{k+1}} \mathbf{A}\, \boldsymbol{\Phi}_s(t_k, t)\, dt \tag{17}$$

The block structure of $\mathbf{A}$ is exactly what the comment block in `orbitdeq()` documents: the upper-right block is $\mathbf{I}$, the middle row block is the acceleration partial matrix `dadx` (dimension $3\times$ `GNX`), and the bottom row is structurally zero because the SRP parameters are random constants.

When the transition matrix is obtained by differencing instead, the propagation is repeated once per state component. Writing the perturbed initial states as $\mathbf{x}_0^{(i\pm)} \equiv \mathbf{x}_0 \pm \delta_i \mathbf{e}_i$, with δ_i the step for component i, the two forms are

$$\Phi_{ij}^{(2p)} = \frac{x_j\left(t_{k+1};\, \mathbf{x}_0^{(i+)}\right) - x_j(t_{k+1};\, \mathbf{x}_0)}{\delta_i} \qquad \Phi_{ij}^{(3p)} = \frac{x_j\left(t_{k+1};\, \mathbf{x}_0^{(i+)}\right) - x_j\left(t_{k+1};\, \mathbf{x}_0^{(i-)}\right)}{2\delta_i} \tag{18}$$

with $\delta_i = 0.001$ for position and velocity and $\delta_i = 10^{-3}$ for the SRP parameters and the clock states. The acceleration partials themselves are obtained either analytically (`ACCLGRAD_ANALYSIS`), by differencing on the total acceleration (`ACCLGRAD_NUMDIFF_ALL`, steps $\delta_r = \delta_v = 0.001$, $\delta_\kappa = 10^{-3}$) or per force component (`ACCLGRAD_NUMDIFF_SEP`):

$$\frac{\partial \mathbf{a}}{\partial x_i} \approx \frac{\mathbf{a}(\mathbf{x} + \delta_i \mathbf{e}_i) - \mathbf{a}(\mathbf{x} - \delta_i \mathbf{e}_i)}{2\delta_i} \tag{19}$$

4.4 Observation Model

Station *b* observes satellite *s*; the measurements are the ionosphere-free pseudorange ρ and carrier phase φ. The receiver ECEF coordinates are rotated into the ECI frame:

$$\mathbf{r}_{b,\mathrm{ECI}}(t_{k+1}) = \mathbf{U}(t_{k+1}) \cdot \mathbf{R}_z(-dt_b \cdot \omega_e) \cdot \mathbf{r}_{b,\mathrm{ECEF}} \tag{20}$$

The signal transmit time is solved iteratively because the satellite moves while the signal travels:

$$t_{k+1}^s = t_{k+1} - \left(\frac{\rho_b^s(t_{k+1})}{c} + dt_s\right) \tag{21}$$

and the satellite state at that transmit epoch is obtained by integrating the equations of motion backwards to t_{k+1}^s:

$$\mathbf{x}_s\left(t_{k+1}^s\right) = \int_{t_{k+1}^s}^{t_{k+1}} \dot{\mathbf{x}}_s\, dt \tag{22}$$

Both frequencies are combined into the ionosphere-free observable before the filter sees them,

$$\rho_b^s = \frac{f_1^2\rho_1 - f_2^2\rho_2}{f_1^2 - f_2^2} \qquad \varphi_b^s = \frac{f_1^2\varphi_1 - f_2^2\varphi_2}{f_1^2 - f_2^2} \tag{23}$$

which is what `estopt.nf = 2` selects inside `corr_meas()`.

The complete measurement path, step by step:

Step	What happens	Equations
1	The filter state at the reception epoch t_{k+1} provides the satellite state $\mathbf{x}_s$	(1), (2)
2	The receiver ECEF position is rotated into the ECI frame with the Earth-rotation matrix and the receiver clock offset	(20)
3	The transmit epoch t_{k+1}^s is solved iteratively from the measured range, the speed of light and the satellite clock offset	(21)
4	The satellite state at that transmit epoch is recovered by integrating the equations of motion back to t_{k+1}^s; the same integration yields $\mathbf{\Phi}_{s_i}(t_{k+1}, t_{k+1}^{s_i})$	(22), (31)
5	The modelled pseudorange and phase, their partials and the tropospheric mapping value are formed	(24) to (30), (32)
6	Prefit residuals and the design matrix are handed to the EKF measurement update	(34) to (36)

Writing $\Delta r_b^s = \|\mathbf{r}_{b,\mathrm{ECI}} - \mathbf{x}_s(t^s)\|$ for the
geometric range, the observation equations for the ionosphere-free pseudorange and carrier phase are

$$\rho_b^s = \Delta r_b^s + T_b^s + c\,(dt_b - dt_s) + \varepsilon_\rho \tag{24}$$

$$\varphi_b^s = \Delta r_b^s + T_b^s + c\,(dt_b - dt_s) + B_b^s + \varepsilon_\varphi \tag{25}$$

The tropospheric delay is split into a hydrostatic part taken from a model and a wet part that is estimated, optionally with horizontal gradients,

$$T_b^s = m_h(el)\,\mathrm{ZHD} + m_w(el)\,(\mathrm{ZTD} - \mathrm{ZHD}) + m_w(el)\cot(el)\,(G_N\cos az + G_E \sin az) \tag{26}$$

where `ZHD` comes from the Saastamoinen model (`tropmodel()`, 70 % relative humidity), the mapping
functions from GMF or NMF (`tropmapf()`) and `ZTD` is the estimated state, which is why the partials with respect to the ZTD and the gradients are just the mapping factors (`trop_model_prec()` returns `dtdx[0] = m_w`).

The estimated parameter vector is

$$\mathbf{X} = \left(\mathbf{x}_{s_1}^{\mathsf{T}}, \ldots, \mathbf{x}_{s_n}^{\mathsf{T}};\;\; dt_{s_1}, \ldots, dt_{s_n};\;\; dt_{b_1}, \ldots, dt_{b_m};\qquad B;\;\; \mathbf{Z}_b^0\right)^{\mathsf{T}} \tag{27}$$

The original renders $\mathbf{X}$ as a single tall column; it is written here as the transpose of a row so that the equation number stays clear of the formula.

The measurement Jacobian is a sparse row vector. With $\mathbf{u}_{b_j}^{s_i}$ the unit line-of-sight of eq. (30) and $\mathbf{\Phi}_{s_i}$ the transition matrix of eq. (31), the pseudorange row is

$$\mathbf{H}_{\rho,b_j}^{s_i} = \left[\cdots\;\; \mathbf{u}_{b_j}^{s_i\,\mathsf{T}}\mathbf{\Phi}_{s_i}\;\; \cdots\;\; -c\;\; \cdots\cdots\;\; +c\;\; \cdots\;\; 0\;\; \cdots\;\; M_{b_j}^{s_i}\right] \tag{28}$$

and the carrier-phase row differs only in the ambiguity entry:

$$\mathbf{H}_{\varphi,b_j}^{s_i} = \left[\cdots\;\; \mathbf{u}_{b_j}^{s_i\,\mathsf{T}}\mathbf{\Phi}_{s_i}\;\; \cdots\;\; -c\;\; \cdots\cdots\;\; +c\;\; \cdots\;\; 1\;\; \cdots\;\; M_{b_j}^{s_i}\right] \tag{29}$$

Jacobian block	Exact expression	Notes
Satellite orbit state	$\dfrac{\partial \rho_{b_j}^{s_i}}{\partial \mathbf{x}_{s_i}(t_{k+1}^{s_i})} \boldsymbol{\Phi}_{s_i}(t_{k+1}, t_{k+1}^{s_i})$	unit line-of-sight rotated from the transmit back to the observation epoch
Satellite clock	$\dfrac{\partial \rho_{b_j}^{s_i}}{\partial dt_{s_i}} = -c$	identical for pseudorange and phase
Receiver clock	$\dfrac{\partial \rho_{b_j}^{s_i}}{\partial dt_{b_j}} = +c$	set to zero for the reference station, eq. (39) and (40)
Ambiguity	0 for pseudorange, 1 for phase	eq. (32)
Troposphere	$M_{b_j}^{s_i}$	mapping function value

The range partial is the unit line-of-sight vector
$\mathbf{u}_{b_j}^{s_i} = \left(\mathbf{r}_{b_j,\mathrm{ECI}} - \mathbf{x}_{s_i}\right) / \left\|\mathbf{r}_{b_j,\mathrm{ECI}} - \mathbf{x}_{s_i}\right\|$,
padded with zeros for the velocity, parameter and clock blocks:

$$\frac{\partial \rho_{b_j}^{s_i}}{\partial \mathbf{x}_{s_i}(t_{k+1}^{s_i})} = \left[\, \mathbf{u}_{b_j}^{s_i\,\mathsf{T}} \;\; \mathbf{0}_{1\times 3} \;\; \mathbf{0}_{1\times \mathrm{GNS}} \;\; \cdots \;\; \mathbf{0}_{1\times 2} \,\right] \tag{30}$$

and the satellite state is mapped from the transmit epoch back to the observation epoch by the transition matrix:

$$\frac{\partial \mathbf{x}_{s_i}(t_{k+1}^{s_i})}{\partial \mathbf{x}_{s_i}(t_{k+1})} = \boldsymbol{\Phi}_{s_i}(t_{k+1},\, t_{k+1}^{s_i}) \tag{31}$$

$$\frac{\partial \rho_{b_j}^{s_i}}{\partial B_{b_j}^{s_i}} = 0, \qquad \frac{\partial \varphi_{b_j}^{s_i}}{\partial B_{b_j}^{s_i}} = 1 \tag{32}$$

The weight of every measurement is elevation dependent. With a and b the two error coefficients (`estopt.err[1]` and `estopt.err[2]`, both 0.003 m in the example),

$$\sigma^2 = (F\,a)^2 + \left(F\,\frac{b}{\sin el}\right)^2 \qquad F = f_{sys}\cdot f_{ratio}\cdot f_{IF}\cdot f_{band} \tag{33}$$

Factor	Value
f_{sys}	1.0 GPS/QZSS, 1.5 GLONASS, 3.0 SBAS (`EFACT_*`)
f_{ratio}	`estopt.eratio[]`, 100 in the example, applied to the code measurements
f_{IF}	3.0 when `ionoopt = IONOOPT_IFLC` (the ionosphere-free combination)
f_{band}	10.0 for GPS/QZSS L5 (`EFACT_GPS_L5`), otherwise 1.0

With the example settings this gives about 0.3 m for the code and 0.003 m for the phase at zenith.

Corrections applied to the modelled observation, and where they live in the code:

Correction	Function	Option
Satellite antenna phase center / variation	`setpcv()`, `satantpcv()`, `satantoff()`	`posopt[0]`, `satantp`
Receiver antenna phase center / variation	`antmodel()`, `setpcv()`	`posopt[1]`, `rcvantp`
Phase wind-up	`model_phw()`	`posopt[2]`
Tidal station displacement	`tidedisp()` (solid bit 1, ocean loading bit 2, pole tide bit 4)	`tidecorr`, `blq`, `odisp`
Troposphere	`model_trop()`, `trop_model_prec()`, mapping function from GMF/NMF	`estopt.tropopt`
Ionosphere (IF combination)	`model_iono()`, `ionocorr()`	`estopt.ionoopt`, `estopt.nf`
Code bias	`uddcb_pod()`, `readdcb()`	`dcb`
Relativistic clock correction	`relcorr()`	always
Earth rotation / Sagnac effect	`ecsf2ecef()`, `satposeci()`, `rcvposeci()`	always

4.5 Filtering: Time and Measurement Updates



At every observation epoch the filter runs the same six steps: carry the state, predict it with the force model, model the measurements, screen them, correct the state, and publish the solution.

Satellite and receiver clocks are modeled as white noise or a first-order Markov process, the IF ambiguity as a random constant and the zenith tropospheric delay as a first-order Markov process. For the satellite clock the code builds the analytic Markov transition explicitly:

$$\begin{aligned} \mathbf{F}_{GIC,GICR} &= \tau\left(1 - e^{-|\Delta t|/\tau}\right) \\ \mathbf{F}_{GICR,GICR} &= e^{-|\Delta t|/\tau}, \qquad \tau = 3600\ \mathrm{s} \\ \mathbf{F}(t_k, t_{k+1}) &= \mathrm{diag}(\mathbf{\Phi}_{s_1}, \ldots, \mathbf{\Phi}_{s_n}, \mathbf{1}, \ldots, \mathbf{1}) \end{aligned} \tag{34}$$

$$\mathbf{x}_{k+1}^{-} = \mathbf{F}_k\,\mathbf{x}_k^{+} \tag{35}$$

Writing $\mathbf{F}_k \equiv \mathbf{F}(t_k, t_{k+1})$, the covariance is propagated as

$$\mathbf{P}_{k+1}^{-} = \mathbf{F}_k\,\mathbf{P}_k^{+}\,\mathbf{F}_k^{\mathsf{T}} + \mathbf{Q}_{k+1} \tag{36}$$

with the process-noise matrix reflecting what each parameter is allowed to do between two epochs,

$$\mathbf{Q} = \operatorname{diag}\!\left(\mathbf{Q}_{sat}[1], \ldots, \mathbf{Q}_{sat}[n],\;\; \mathbf{Q}_{clk}^{s},\; \mathbf{Q}_{clk}^{b},\; \mathbf{0}_B,\; \mathbf{Q}_{ztd}\right) \tag{37}$$

Block	Model	Settings in the example
Orbit, position / velocity	random walk (white noise)	`satvar` = 5 m² / (0.1 mm/s)²
SRP and empirical parameters	random constant with small process noise	`satvar` = 0.1², `satprn` = 1e-15
Satellite clock	white noise or first-order Markov, $\tau = 3600$ s	`satvar` = 3 m², `satprn` = 1.0
Receiver clock	white noise, one state per system	`rcvvar` = 60 m²
Ambiguity	random constant, $\mathbf{Q} = \mathbf{0}$	initial variance 60² m²
ZTD	first-order Markov	`rcvvar` = 0.5 m²

The satellite clock transition above is the difference form of

$$\mathbf{Q}_{clk}^{s} = \sigma_{clk}^{2}\left(1 - e^{-2|\Delta t|/\tau}\right) \tag{38}$$

With station b_R as reference clock, its clock parameters are removed from the estimation:

$$\frac{\partial \rho_{b_R}^{s_i}}{\partial dt_{b_R}} = 0 \tag{39}$$

$$\frac{\partial \varphi_{b_R}^{s_i}}{\partial dt_{b_R}} = 0 \tag{40}$$

The measurement update follows the standard Kalman form, with $\mathbf{R}$ the diagonal matrix of the variances of eq. (33) and $\mathbf{v}$ the prefit residual vector:

$$\mathbf{K} = \mathbf{P}_{k+1}^{-}\mathbf{H}^{\mathsf{T}}\left(\mathbf{H}\mathbf{P}_{k+1}^{-}\mathbf{H}^{\mathsf{T}} + \mathbf{R}\right)^{-1} \tag{41}$$

$$\mathbf{x}_{k+1}^{+} = \mathbf{x}_{k+1}^{-} + \mathbf{K}\,\mathbf{v}, \qquad \mathbf{v} = \mathbf{z} - h\!\left(\mathbf{x}_{k+1}^{-}\right) \tag{42}$$

$$\mathbf{P}_{k+1}^{+} = (\mathbf{I} - \mathbf{K}\mathbf{H})\,\mathbf{P}_{k+1}^{-} \tag{43}$$

This is exactly what `filter()` in `rtkcmn.c` implements — a dense, non-square-root formulation with the states stored column-major.

A priori constraints. The library can additionally constrain the satellite orbit and clock to the a priori ephemeris by appending pseudo-observations with an identity design matrix,

$$\mathbf{v}_c = \mathbf{r}_{\text{eph}}^{\text{ECI}} - \mathbf{r}_{s}^{\text{ECI}}, \qquad \mathbf{H}_c = \mathbf{I} \qquad \sigma^2 = \begin{cases} (5\ \text{m})^2 & \text{position} \\ (1\ \text{mm/s})^2 & \text{velocity} \end{cases} \tag{44}$$

implemented by `constsatposveleph()` and `constsatclk()` (clock: 5 m and 1e-5 m/s). Both are behind

the compile-time switches `CONSTEPH_POSVEL` and `CONSTEPH_CLK`, which are **0 by default**, so the shipped build estimates the orbit from the GNSS observations alone.

Smoothing and combination. Given the forward solution (x_f, P_f) and the backward solution (x_b, P_b) of the same state, `smoother()` forms the inverse-covariance weighted mean,

$$\mathbf{P}_s = \left(\mathbf{P}_f^{-1} + \mathbf{P}_b^{-1}\right)^{-1} \qquad \mathbf{x}_s = \mathbf{P}_s \left(\mathbf{P}_f^{-1}\mathbf{x}_f + \mathbf{P}_b^{-1}\mathbf{x}_b\right) \tag{45}$$

which is what `podsatorbfitfbsprc()` applies after matching the two passes epoch by epoch. The alternative RTS recursion, used by `podsatorbfitrtsprc()` and `rts()`, runs backwards through the stored forward solutions,

$$\begin{aligned} \mathbf{G}_{k-1} &= \mathbf{P}_{f,k-1}\mathbf{F}_{k-1}^{\mathsf{T}}\left(\mathbf{P}_{p,k}\right)^{-1} \\ \mathbf{x}_{s,k-1} &= \mathbf{x}_{f,k-1} + \mathbf{G}_{k-1}\left(\mathbf{x}_{s,k} - \mathbf{x}_{p,k}\right) \\ \mathbf{P}_{s,k-1} &= \mathbf{P}_{f,k-1} + \mathbf{G}_{k-1}\left(\mathbf{P}_{s,k} - \mathbf{P}_{p,k}\right)\mathbf{G}_{k-1}^{\mathsf{T}} \end{aligned} \tag{46}$$

where the subscript p marks the forward-predicted quantities.

4.6 Processing Strategy



The shipped example reads the whole observation stream forward, rewinds the binary file and filters it again in reverse time order. The two passes are then differenced against the precise ephemeris and

statistics are formed per satellite, which is a cheap way of exposing model errors: the forward pass is dominated by the a priori orbit early in the arc and by its own dynamics late in the arc, and the backward pass shows the mirror image.

Alternative strategies available inside the library:

Entry point	Strategy
`podflt()` in a forward loop	Forward filtering only
`podflt()` forward + backward loop (the example)	Bidirectional filtering with independent statistics
`podfbcmb()` / `podfbcmbe()`	Forward/backward combined filter over the binary observation file, using eq. (45)
`podsatorbfit(type, ...)`	Single-satellite orbit fit, `type = 0` forward/backward combination, `type = 1` RTS smoother (eq. (46))

4.7 Mapping to `gpod.pdf`

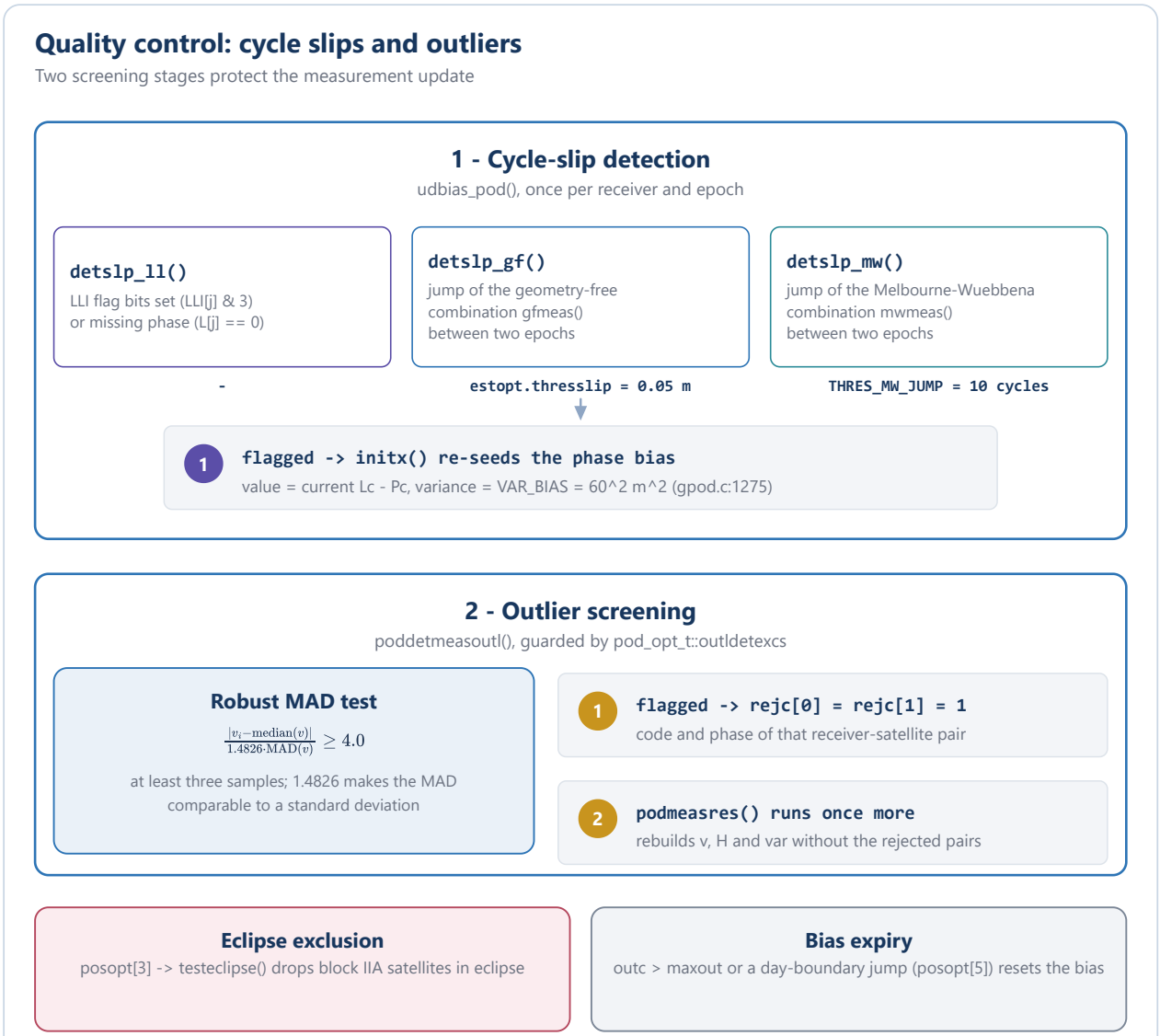
`gpod.pdf`	Here	Content
(1)	(1)	Satellite state vector
(2)	(2)	Equations of motion
(3)	(3)	Acceleration decomposition
(4)	(4)	Numerical propagation
(5)	(14)	State transition matrix
(6)	(15)	Variational equation
(7)	(16)	Design matrix A
(8)	(17)	Transition matrix by integration
(9)	(20)	Receiver ECEF to ECI
(10)	(21)	Transmit time
(11)	(22)	Satellite state at the transmit epoch
(12)	(24)	Pseudorange observation equation
(13)	(25)	Carrier-phase observation equation
(14)	(27)	Estimated parameter vector
(15)	(28)	Jacobian of the pseudorange
(16)	(29)	Jacobian of the phase
(17)	(30)	Range partial

`gpod.pdf`	Here	Content
(18)	(31)	State mapping by the transition matrix
(19)	(32)	Ambiguity partials
(20)	(34)	Overall transition matrix
(21)	(36)	Covariance prediction
(22)	(39)	Reference-clock partial, pseudorange
(23)	(40)	Reference-clock partial, phase

Equations (5) to (13), (18) to (19), (23), (26), (33), (35), (37) to (38) and (41) to (46) are not numbered in `gpod.pdf`; they are listed here because the implementation depends on them.

4.8 Quality Control: Cycle Slips and Outliers

The "screen them" step of section 4.5 is two separate mechanisms: phase-bias re-initialization after cycle slips, which runs inside the time update, and a robust outlier test on the prefit residuals, which runs just before the measurement update. All of the bookkeeping lives in the per-receiver, per-satellite table `flt.ssat[rcv][sat]`.



Cycle-slip detection. `udbias_pod()` runs three detectors for every receiver and epoch:

Detector	Test	Threshold
`detslp_ll()`	LLI flag bits set (`LLI[j] & 3`) or missing phase (`L[j] == 0`)	— (per frequency)
`detslp_gf()`	epoch-to-epoch jump of the geometry-free combination `gfmeas()`	`estopt.thresslip` (0.05 m)
`detslp_mw()`	epoch-to-epoch jump of the Melbourne-Wübbena combination `mwmeas()`	`THRES_MW_JUMP` = 10 cycles

A flagged satellite has its phase-bias state re-initialized from the current `Lc - Pc` value with variance `VAR_BIAS` = 60^2 m^2 (`initx()` , gpod.c:1275) — one epoch later the filter is fully recovered instead of diverging.

Bias expiry. The outage counter `ssat.outc[f]` increments every epoch without a usable observation; once it exceeds `estopt.maxout` (5 epochs in the default `prcopt_default_ppp`), or a

day-boundary clock jump is flagged (`posopt[5]`, detected as GPST rounded to 0.1 s hitting 00:00:00.0), the bias state is reset to zero so that the next observation starts a fresh ambiguity.

Measurement outliers. With `pod_opt_t::outldetexcs` set, `poddetmeasoutl()` collects the code and phase residuals of every valid receiver for each satellite and applies a robust MAD test (`maddetoutl()`):

$$\frac{|v_i - \mathrm{median}(v)|}{1.4826 \cdot \mathrm{MAD}(v)} \geq 4.0$$

The 1.4826 factor makes the median absolute deviation comparable to a standard deviation under normality, and the test needs at least three samples. Flagged receiver–satellite pairs are marked `rejc[] = 1`; when anything was flagged, `podmeasres()` rebuilds the prefit residuals, the design matrix and the variances once without them, so the update never sees a rejected measurement.

Eclipse exclusion. With `posopt[3]` set, `testeclipse()` drops block IIA satellites in eclipse before the residuals are formed — their yaw attitude is unreliable during eclipses, which corrupts both the antenna offset and the SRP modelling.

`ssat[rcv][sat]` field	Meaning
`slip[NFREQ]`	cycle-slip flag of the current epoch, per frequency
`outc[NFREQ]`	epochs since the last usable observation, per frequency
`rejc[2]`	code / phase rejected by the MAD test
`gf`, `mw`	previous geometry-free and MW combination values
`resp`, `resc`	phase and code residuals of the epoch
`vsat[NFREQ]`	measurement valid after all screening
`azel[2]`	azimuth / elevation, drives the weighting of eq. (33)

Thresholds and switches: `thresslip`, `maxout`, `outldetexcs` in 9.2; new rules hook in at `poddetmeasoutl()` / `maddetoutl()` or at the `detslp_*()` family — see the extension table in 6.6.

---

5. Repository Layout

Path	Description
`CMakeLists.txt`	Top-level build: bundled dependencies → static library `gpod` → `test_podflt`
`README.md`	This document
`gpod.pdf`	Algorithm description (Chinese): state equations, variational equations, observation model

Path	Description
`gpod-en.md`	The algorithm description in English: the same 23 equations as `gpod.pdf`, written out in LaTeX
`gpod-en.pdf`	The same document as a 5-page A4 PDF, with the formulas typeset by KaTeX
`test_podflt.c`	Minimal working example: forward + backward filtering with residual statistics
`src/`	All POD source code (11 `.c` and 3 `.h` files, about 18 900 lines)
`assets/`	The nine standalone SVG figures used in this document
`lib/sofa/`	SOFA release 2023-10-11 — IAU standards of fundamental astronomy
`lib/nrlmsise/`	NRLMSISE-00 density model, `eopspw` EOP/space-weather reader, and sample `EOP-All.txt` / `SW-All.txt`
`lib/openblas/`	OpenBLAS 0.3.27 — BLAS / LAPACK / CBLAS
`lib/gsl/`	GNU Scientific Library 2.7 — ODE solvers (`ode-initval2`), error functions
`lib/boost/`	Boost 1.87.0 — only `Numeric.Odeint` is used
`lib/oceantide/`	`fes2004_Cnm-Snm.dat` — ocean tide potential coefficients

Runtime data (`data/model`, `data/nav`, `data/obs`) is not distributed with the repository; see section 10.

---

6. Code Structure

6.1 Module Responsibilities

Line counts follow `wc -l`, so a file that does not end in a newline is reported one line short of its logical length.

File	Lines	Layer	Responsibility
`src/rtklib.h`	1197	Interface	Constants, enums, structs, prototypes; also hosts the constellation switches (`ENAGLO`, `ENACMP`, ...)
`src/podlib.h`	463	Interface	The POD API: `pod_t`, `pod_opt_t`, force-model structs, state indices, all prototypes
`src/rtkcmn.c`	3745	Base	Timescales, coordinate transforms, matrix/vector algebra, dense `filter()` / `smoother()`, LAPACK wrappers, EOP/ATX/BLQ/SNX/LEAPS readers, statistics
`src/rinex.c`	2974	Data I/O	RINEX 2/3 observation, navigation (GPS/GLO/GAL/QZS/BDS/IRN) and clock read/write, observation-code indexing

File	Lines	Layer	Responsibility
`src/gint.c`	3054	**Core**	Force model, shadow models, the nine integrators, variational equations, transition matrix, IERS diurnal/sub-diurnal polar-motion terms
`src/gpod.c`	2237	**Core**	EKF POD: initialization, state indexing, time update, residual/Jacobian assembly, cycle slips, outlier rejection, measurement update, orbit fitting
`src/gcmn.c`	1780	POD utilities	Time-scale conversion, DE440 ephemeris, ECI/ECEF/ECSF/SATF frames, `gsatinfos`, model-file parsers, observation containers, SINEX handling
`src/sbas.c`	893	Augmentation	SBAS message decoding (types 1/2/6/7/9/18/24/25/26) and corrections
`src/ephemeris.c`	801	Data	Broadcast ephemeris and clocks for all supported systems, SBAS and SSR
`src/preceph.c`	660	Data	SP3 reading/interpolation, DCB, satellite antenna offsets, `peph2pos()` / `pephclk()`
`src/pntpos.c`	641	Auxiliary	Standard point positioning, RAIM FDE — for initial values
`src/tides.c`	283	Auxiliary	Station tidal displacement: solid earth tide, ocean loading, pole tide
`src/log.c` / `src/log.h`	114 / 28	Base	Leveled logging; empty stubs when `OUTPUT_LOG = 0`

6.2 Module API Index

Interface exports from `podlib.h`, grouped by purpose.

Time and frames (`gcmn.c`)

Function	Purpose
`cal2mjd()`, `mjd2cal()`	Calendar ↔ MJD
`mjdtdb()`, `mjdtt()`, `mjdut1()`, `reftimediff()`	TDB / TT / UT1 transforms and reference-time differences
`time2mjd()`, `time2mjdtt()`, `time2mjdut1()`, `doy()`	`gtime_t` → MJD / TT / UT1 / day-of-year
`ecsf2ecef()`, `ecsf2ecef2()`, `eci2ecef2()`	ECSF/ECI → ECEF rotation matrices and their time derivatives
`satposeci()`, `satposecef()`	Satellite position ECEF ↔ ECI

Function	Purpose
`rcvposeci()`	Receiver position ECEF → ECI including clock offset rotation
`ecef2satf()`, `ecsf2satf()`, `satf2ecsf()`, `satfixed()`	Satellite body-fixed frame construction
`ssrange()`	Signal-transmission-time-corrected range and line-of-sight partials
`relcorr()`	Relativistic clock correction
`pmut1_ocean()`, `utlibr()`, `pmsdnut2()`	IERS 2010 diurnal/sub-diurnal polar motion, UT1 and LOD corrections

Force model and integration (`gint.c`)

Function	Purpose
`forcemdlinit()`, `forcemdlfree()`	Build / release all sub-models of `force_model_t`
`satforce()`	Sum all force components and their partials
`gravityforce()`, `solidtidecorr()`, `oceantidecorr_*()`, `poletidecorr()`, `rotcorr()`	Gravity field and tidal corrections
`solradpforce()`, `solradmdl_deft()`, `solradmdl_ecom()`, `solradmdl_code()`	SRP families
`atmdragforce()`, `atmdenshpm()`, `atmdensnrl()`, `shadow()`	Drag and shadow
`nbpertnforce()`, `pointmassforce()`, `relattyforce()`	Third body and relativity
`satorbitinit()`, `satorbit()`, `satorbite()`, `satorbitfree()`	Orbit propagation (the `e` variant restores the previous state)
`orbitgetx()`, `orbitsetx()`	Get/set the propagator state vector
`orbintj2()`, `orbintj2_U()`	Lightweight RK4 propagator with a precomputed rotation matrix
`rkf78m()`	Built-in Runge-Kutta-Fehlberg 7(8) with adaptive step
`doodsonargs()`, `legendre()`	Doodson arguments for tides, associated Legendre functions
`upateerpv()`	Refresh the EOP vector from an `erpd_t`

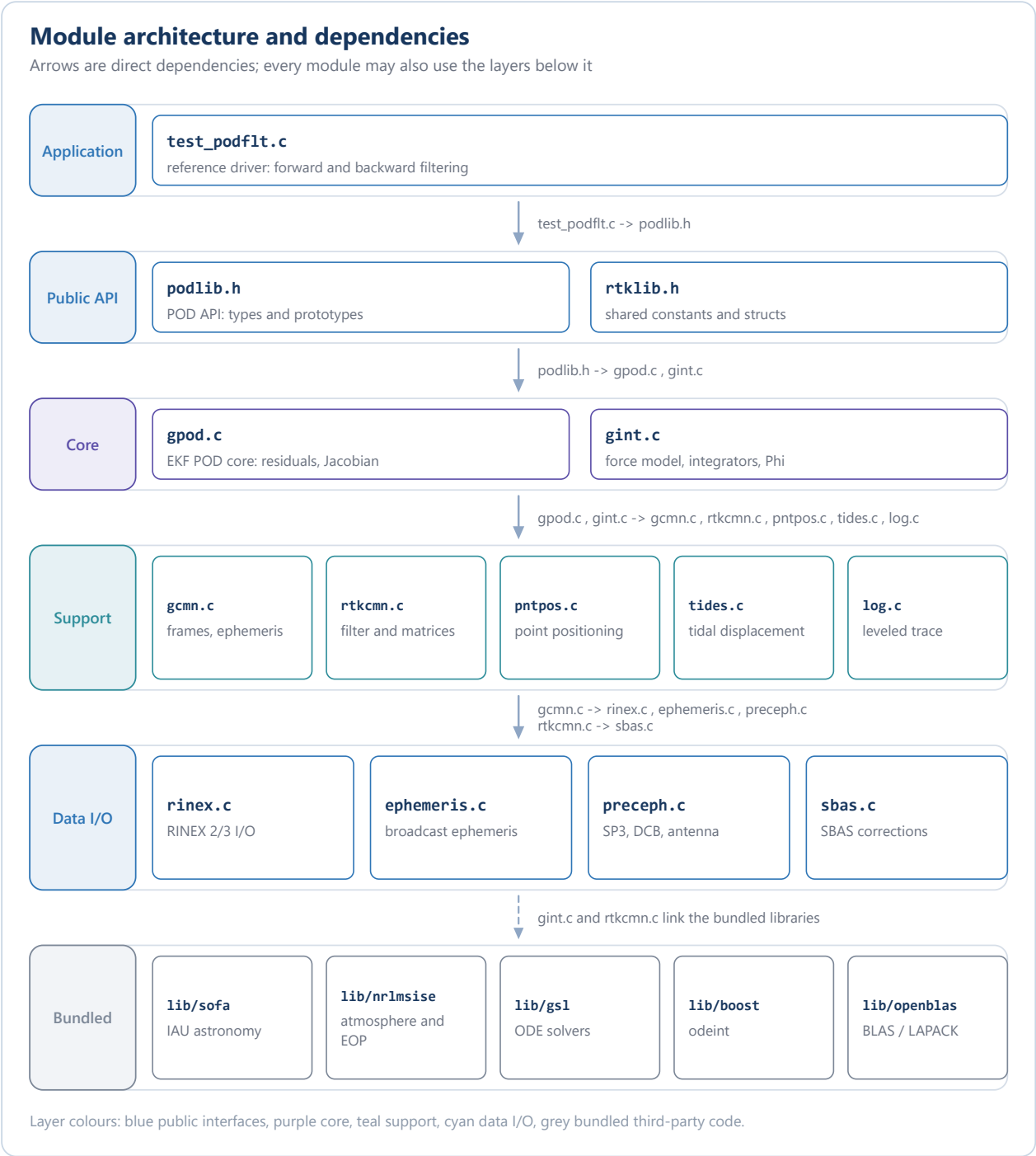
Model and file readers (`gcmn.c`, `rtkcmn.c`, `preceph.c`, `rinex.c`)

Function	Reads
`readgfc()`	Gravity field coefficients (`.gfc`)
`readeop()`, `geteop()`	Earth orientation parameters
`readspw()`	Space weather (F10.7 / Ap / Kp)
`readotfile()`	Ocean tide potential coefficients
`readjplde440coeffs()`	JPL DE440 Chebyshev coefficients
`readrnx()`, `readrnxt()`, `readrnxc()`	RINEX observation / navigation / clock
`readsp3()`, `readsap()`, `readddcb()`	SP3, satellite antenna parameters, DCB
`readpcv()`	Antenna phase center variations (ATX)
`readblq()`, `readpos()`	Ocean loading displacement, station positions
`readsnx()`	Station coordinates from SINEX
`readpodobs_rnx()`, `readpodobs_sp3()`, `sortpodobs()`, `podobssget()`	POD observation container

POD control (`gpod.c`)

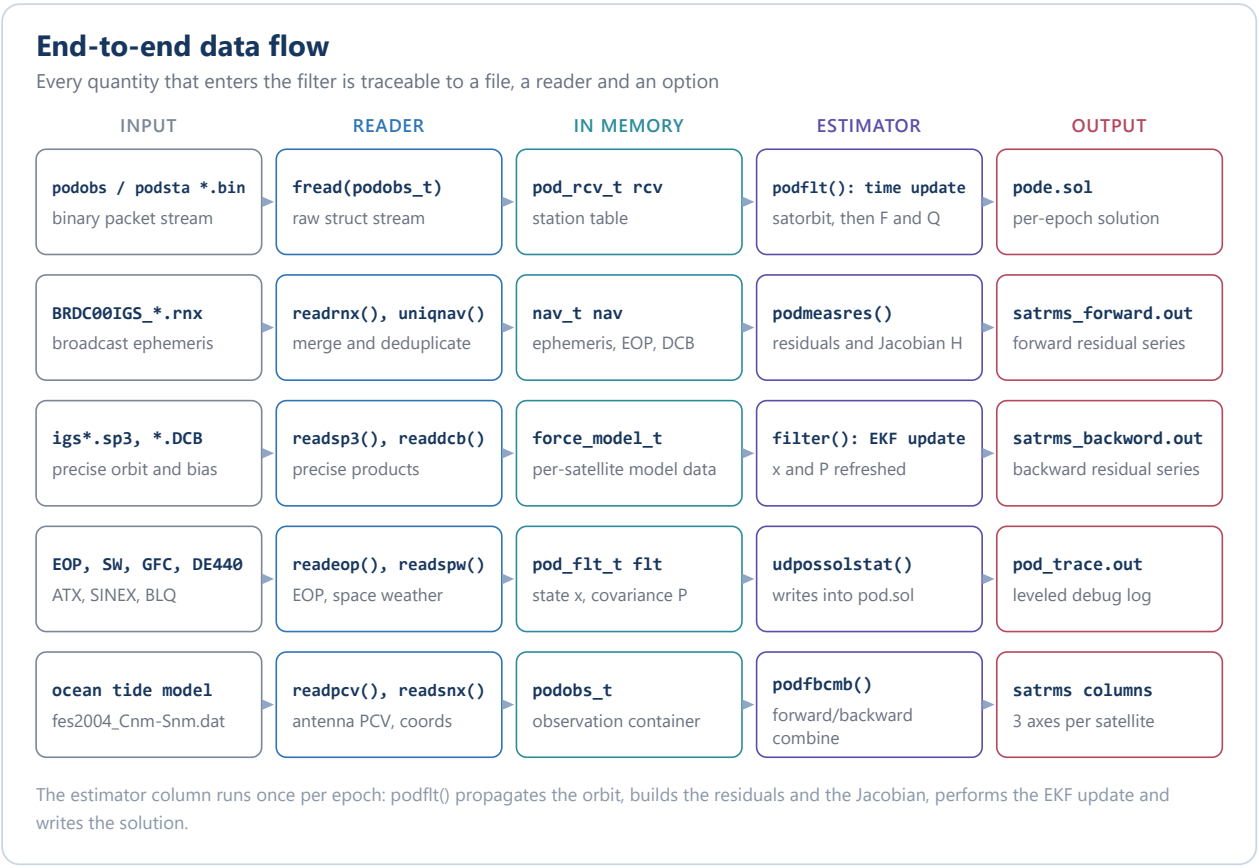
Function	Purpose
`podinit()`, `podfree()`	Initialize / release the complete POD state
`podflt()`	One EKF epoch (dispatches on the observation type)
`podsatposflt()`, `podsatinit()`	Satellite-position-mode filtering and state initialization
`podsatorbfit()`	Single-satellite orbit fit (forward/backward or RTS)
`podudrcvsnx()`, `podsetclkref()`	Station coordinates from SINEX, reference-clock selection
`podfbcmb()`, `podfbcmbe()`	Forward/backward combined filtering from the binary observation file
`GIX*()` family	State index computation for satellites, receivers, biases and DCB

6.3 Module Architecture and Dependencies



The figure shows the direct dependency edges between modules and the layer each file belongs to; individual responsibilities are listed in section 6.1.

6.4 End-to-End Data Flow



6.5 Naming and Coding Conventions

Convention	Detail
Function naming	`snake_case`, RTKLIB style; `extern` / `static` written explicitly at the definition
Module prefixes	`pod*` (POD), `sat*` (satellite), `rcv*` (receiver), `read*` / `out*` (I/O), `detslp*` (cycle slip), `ud*` (update)
Index helpers	`GIX*` family in `podlib.h` — never hard-code state offsets
Constant macros	`GPOD_*`, `ORBITINT_*`, `SOLRAD_MODEL_*`, `ATMODENSITY_*`, `SHADOW_*`, `GRAVITY_MODEL_*`, `UD*`, `ACCLGRAD_*`
Header hygiene	Guard macros plus `extern "C"` wrapping; C++ can include the headers directly
Visibility	Everything not in `podlib.h` / `rtklib.h` is `static`
Logging	`log_trace(level, ...)` with levels 1 error, 2 warning, 3 info, 4 matrix dumps, 5 verbose

6.6 Extension Guide

Goal	Steps
Add a constellation	Define the corresponding `ENA*` macro (see 9.4) so that `MAXSAT` covers it, add the satellites to `gsatinfos[]` in `gcmn.c` (PRN, block type, yaw mode, mass, yaw rate), and make sure the system bit is set in `estopt.navsys` and in `indsys()`
Add an SRP model	Add a `SOLRAD_MODEL_*` constant in `podlib.h`, extend `solrad_pressure_t` if new parameters are needed, implement the force and its partials next to `solradmdl_ecom()`, and dispatch from `solradpforce()`
Add an integrator	Add an `ORBITINT_*` constant, add a `case` in `satorbitprc()`, and either wrap a new GSL/odeint stepper or reuse `orbintj2()` for a fixed-step RK method
Add a model file	Write a `read*()` parser, add the path field to `force_model_opt_t` or `pod_opt_t`, call it from `forcemdlinit()` (force models) or `podinit()` (POD-level data), and add `noneedfree*` bookkeeping if it can be shared
Add a state parameter	Extend the `GNX` / `GNRX` layout and its `GIX*()` accessor, initialize it in `initx()` / `podsatinit()`, map it into `F` and `Q`, and add its Jacobian column in `podmeasres()`
Add a quality-control rule	Hook into `poddetmeasoutl()` / `maddetoutl()` for measurement-level tests, or into `detslp_*()` for phase-level tests

---

7. Core Data Structures

7.1 Top-Level Container

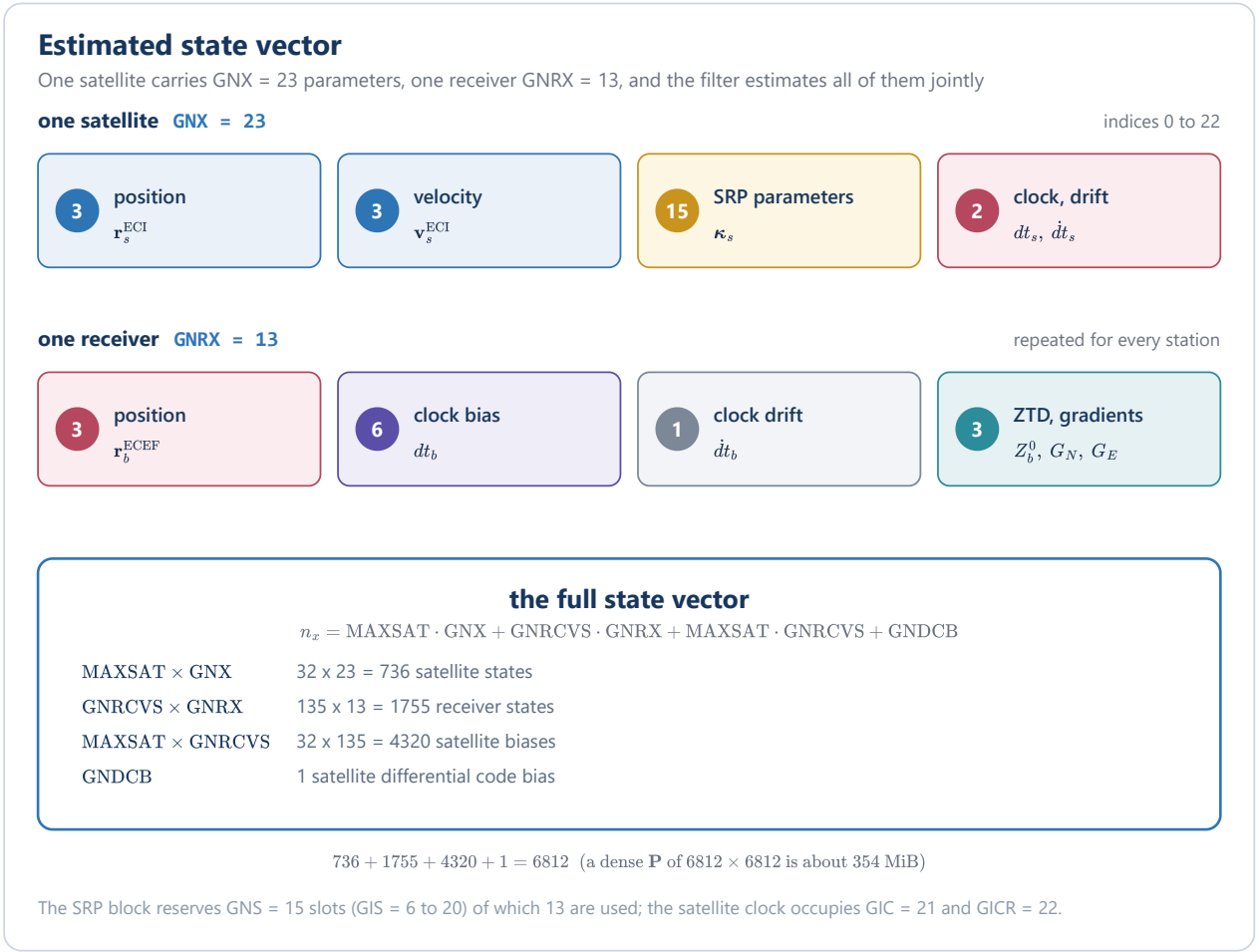
```
typedef struct pod {
    pod_flt_t  flt;             /* filter: state x/P, covariance, per-obs statistics ssat, solut
    pod_sol_t  sol;             /* current epoch solution: xs[sat][GNX], xr[rcv][GNRX], DOP */
    pod_opt_t  opt;             /* processing options: force model, estimation, file paths */
    pod_rcv_t  rcv[GNRCVS];     /* station table: coordinates, antenna type, clock-reference fla
    satorbit_t orbits[MAXSAT];  /* per-satellite integrator state and force-model instance */
    nav_t      nav;             /* broadcast/precise ephemeris, DCB, EOP, ... */
    pcvs_t     pcvss, pcvsr;    /* satellite and receiver antenna phase center variations */
} pod_t;
```

7.2 Filter, Solution and Observation Containers

Struct	Key fields	Notes
`pod_flt_t`	`time`, `dt`, `nx`, `state`, `x`, `P`, `xp`, `Pp`, `x0`, `P0`, `ssat[GNRCVS][MAXSAT]`, `sols[GNRCVS]`, `ismap`, `ibmap`, `nbias`, `clkref[NSYS]`	`x` / `P` are the EKF state and covariance; `xp` / `Pp` hold the predicted (time-updated) values; `ssat[rcv][sat]` keeps per-station-per-satellite residuals, slip flags and phase wind-up

Struct	Key fields	Notes
`pod_sol_t`	`tutc`, `stat`, `ns[MAXSAT]`, `xs[MAXSAT][GNX]`, `xr[GNRCVS][GNRX]`, `Qs`, `Qr`, `dops[MAXSAT]`	One solution epoch; `Qs` / `Qr` are variances of the satellite / receiver states, `dops` is per-satellite DOP
`podobs_t`	`type`, `nrcv`, `nobs[GNRCVS]`, `ircv`, `name`, `obs[GNRCVS][MAXOBS]`, `pos[MAXSAT][6]`, `vel[MAXSAT][6]`, `tutc`	One epoch of binary input; `type` selects the estimation mode, `pos` / `vel` carry satellite positions in mode A
`podobss_t`	`n`, `nmax`, `data[]`, `obss[GNRCVS]`, `stas[GNRCVS]`	Time-ordered observation container used by orbit fitting and combined filtering
`satorbit_t`	`tutc`, `sat`, `state`, `x[GNX]`, `F[GNX*GNX]`, `fmdl`	Propagator state: `x` holds position/velocity plus SRP and clock states, `F` the transition matrix, `fmdl` the owned force model
`pod_rcv_t`	`id`, `clkref`, `name`, `pos[3]`, `sta`	Station record: ECEF coordinates, receiver/antenna metadata, reference-clock flag

7.3 Estimated State Vector



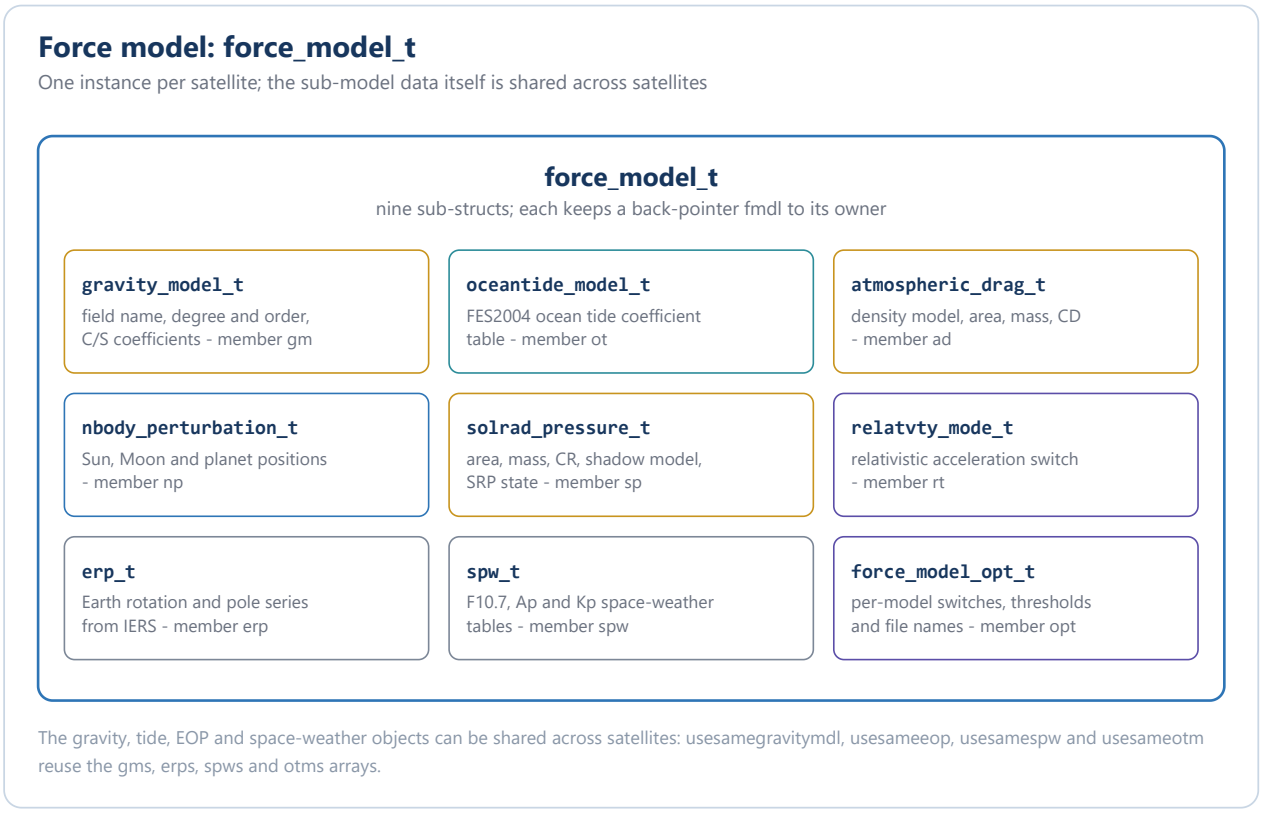
SRP parameter slots: `GNS = 15` slots starting at `GIS = 6`, of which 13 are used (`GIS_D0 = 6` … `GIS_BS = 18`); the clock states follow at `GIC = 21` and `GICR = 22`.

The block layout, from `podlib.h`:

Block	Constant	Size	First index
Satellite position	`GNP`	3	`GIP` = 0
Satellite velocity	`GNV`	3	`GIV` = 3
SRP parameters	`GNS`	15 (13 used, `GIS_D0` to `GIS_BS`)	`GIS` = 6
Satellite clock and drift	`GNC`	2	`GIC` = 21
Receiver position	`GNRP`	3	`GIRP` = 0
Receiver clock bias, one per system	`GNRC`	6	`GIRC` = 3
Receiver clock drift	`GNRCR`	1	`GIRD` = 9
Receiver ZTD and gradients	`GNRTR`	3	`GITR` = 10

Accessors: `GIXSAT_POS()` / `GIXSAT_VEL()` / `GIXSAT_SRP()` / `GIXSAT_CLK()` / `GIXSAT_CKR()` / `GIXRCV_POS()` / `GIXRCV_CLK()` / `GIXRCV_TRP()` / `GIXSAT_BIAS()` / `GIXDCB()`. The last two are **not** inside the per-satellite or per-receiver blocks: `GIXSAT_BIAS()` returns `MAXSAT*GNX + GNRCVS*GNRX + MAXSAT*(rcv-1) + sat-1` and `GIXDCB()` returns the final element `MAXSAT*GNX + GNRCVS*GNRX + MAXSAT*GNRCVS`.

7.4 Force Model Structure



`force_model_t` holds nine sub-structs; every one keeps a back-pointer `fmdl` to its owner, so the force routines can reach the shared EOP, space-weather and ephemeris data without extra

arguments.

Member	Type	Holds
`gm`	`gravity_model_t`	Field name, degree and order, tide switches, `C` / `S` coefficients
`ot`	`oceantide_model_t`	Ocean tide coefficient table (FES2004)
`ad`	`atmospheric_drag_t`	Density-model option, area, mass, `CD`
`np`	`nbody_perturbation_t`	Sun, Moon and planet positions
`sp`	`solrad_pressure_t`	Area, mass, `CR`, shadow option, SRP model and state
`rt`	`relatvty_mode_t`	Relativistic acceleration switch
`erp`	`erp_t`	Earth rotation and polar-motion series
`spw`	`spw_t`	F10.7, Ap and Kp space-weather tables
`opt`	`force_model_opt_t`	Per-model switches, thresholds and file names

Sharing across satellites is controlled by `usesamegravitymdl` / `usesameeop` / `usesamespw` / `usesameotm`, and the `gms` / `erps` / `spws` / `otms` pointer arrays hold the shared instances.

7.5 Satellite Metadata Table `gsatinfos[256]`

Defined in `gcmn.c`, indexed by satellite number and retrieved with `podgetsatinfo(sat)`:

Field	Meaning	Used by
`id`	PRN string, e.g. `G01`, `C30`, `E24`	logging, SP3/RINEX matching
`blocktype`	Satellite type, e.g. `BLOCK IIIA`	copies into `solrad_pressure_t::blocktype` for model-specific SRP handling
`yaw_bias`	Yaw-attitude bias flag (`N` / `Y`)	yaw model in `sat_yaw()`
`mass`	Mass in kg (e.g. 2161.0 for G01)	assigned to `solrad_pressure_t::mass` by `podinit()`
`yaw_rate`	Nominal yaw rate	yaw model

The table currently covers GPS, BDS, Galileo, GLONASS, QZSS and IRNSS satellites.

---

8. Key Workflows and Call Chains

8.1 Initialization

podinit() initialization call chain

The order matters: the force models must exist before the first propagation

podinit(pod, opt)

one call, in this order

1. **`setrcvs()`**
   receiver options: ids, and the clock-reference flag
2. **`readstapos()`**
   station coordinates into the receiver table
3. **`podsetclkref()`**
   marks which station provides the reference clock
4. **`readrnx(), uniqnav()`**
   broadcast ephemeris for navfile and navfile2, merged
5. **`readpcv(), readdcb()`**
   satellite and receiver antenna PCV, then the DCBs
6. **`satorbitinit()`**
   per satellite: forcemdlinit() loads every force model file
7. **`state allocation`**
   flt.x, flt.P, flt.xp, flt.Pp = zeros(nx), nx = 6812
8. **`pod->opt = *opt`**
   the option set is stored; podfree() releases it all later

satorbitinit() calls forcemdlinit() and then seeds the 13 SRP state parameters; gsatinfos[] supplies the SRP mass and block type, while the drag mass stays at atmdrgparas[1].

`podinit()` runs these steps in order:

1. `setrcvs()` — receiver options, ids and the clock-reference flag;
2. `readstapos()` — station coordinates into the receiver table;
3. `podsetclkref()` — marks the station that provides the reference clock;
4. `readrnx()` for `navfile` and `navfile2`, then `uniqnav()` — broadcast ephemeris, merged;
5. `readpcv()` for the satellite and receiver antenna files, then `readdcb()`;
6. `satorbitinit()` per satellite — this calls `forcemdlinit()`, which loads every force-model file, and then seeds the 13 SRP state parameters; `gsatinfos[]` supplies the SRP mass and block type;

7. state allocation — `flt.x`, `flt.P`, `flt.xp` and `flt.Pp` are `zeros()` of size `nx = 6812`, and the run aborts with `no enough memory` if any allocation fails;
8. `pod->opt = *opt`; `podfree()` releases everything when the run ends.

Three consequences worth knowing:

- `pod->flt.nx` is the **fixed dimension reserved for the maximum number of satellites and stations**, not the number of parameters actually estimated. `P` is a dense square matrix sized `nx × nx` — see 10.3 for the numbers.
- `podinit()` overrides `solrad_pressure_t::mass` and `blocktype` from `gsatinfos[]`, but the **drag mass stays at** `atmdrgparas[1]` from the options (8000 kg in the example). If you use a per-satellite mass, set `atmdrgparas[1]` per satellite in `fmdlopts[sat]`.
- Precise ephemeris is loaded by the caller (the example reads three days of SP3 after `podinit()`); `podfbcmb()` instead drives everything from `obsbinfile`.

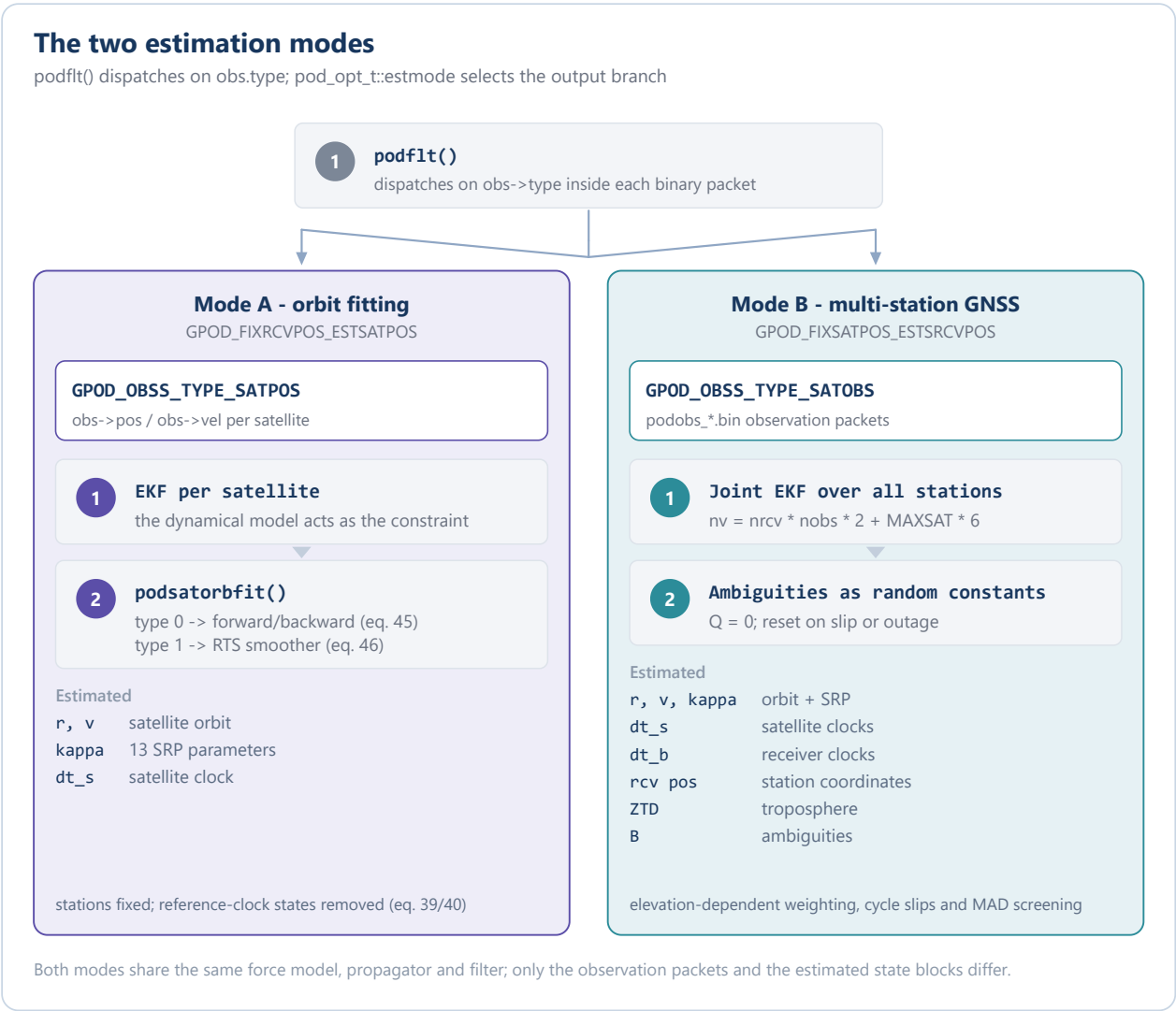
8.2 Per-Epoch Filtering

The per-epoch pipeline is shown in Figure 2; in mode B the measurement vector is dimensioned before the update:

$$n_v = n_{rcv} \cdot n_{obs}^{max} \cdot 2 + MAXSAT \cdot 6$$

where n_{obs}^{max} is the largest number of observations contributed by any single station in that epoch (bounded by `MAXOBS = 64`), and the factor 2 accounts for pseudorange plus phase. The code allocates a **dense** design matrix `H` of size $n_v \times$ `nx` and a dense weight matrix `R` of size $n_v \times n_v$ for every epoch, then frees them. Quality control sits between the residual assembly and the measurement update — see section 4.8.

8.3 The Two Estimation Modes



Mode	Observation type	`estmode`	Behavior
A — satellite-position mode	`GPOD_OBSS_TYPE_SATPOS` (`obs->pos/vel`)	`GPOD_FIXRCVPOS_ESTSATPOS`	Station coordinates fixed; observations are satellite positions/velocities and the dynamical model acts as the constraint, i.e. **orbit fitting**. `podsatorbfit()` provides forward/backward and RTS variants

Mode	Observation type	`estmode`	Behavior
B — station-observation mode	`GPOD_OBSS_TYPE_SATOBS` (multi-station GNSS observations)	`GPOD_FIXSATPOS_ESTSRCVPOS`	Multi-station joint EKF: estimates **satellite orbit + SRP + satellite clocks** together with **receiver coordinates + clock offsets + ZTD**, ambiguities as random constants

`podflt()` dispatches on `obs.type` (stored inside each binary observation packet), while `pod_opt_t::estmode` selects the output branch and the interpretation of the solution arrays.

8.4 Propagator and Transition Matrix

Aspect	Value in the shipped code
Integrated state (variational mode)	$6 + \mathrm{GNX}^2 = 6 + 529 = 535$ elements: position, velocity and the full transition matrix
Integrated state (numerical-differential mode)	6 elements (position, velocity only)
GSL `odeiv2` driver	standard adaptive driver, initial step 0.01 s, absolute and relative tolerance `1E-10`, $\mathrm{hmax} = \max(\lvert dt \rvert, 0.01)$
Built-in `rkf78m()`	initial step 0.01 s, tolerance `1E-10`
`ACCLGRAD_NUMDIFF_ALL` steps	`dr = dv = 0.001`, SRP parameter step `dp = 1E-3`
`UDTRANSMATRIX_NUMDIF_*` steps	0.001 for position/velocity, `1E-3` for SRP parameters and clock states
Satellite clock Markov correlation time	`corrtime = 3600 s`
Non-destructive trial propagation	`satorbite()`, which restores `x`, `F`, `tutc` and `state` afterwards
Fallback	`satorbitprc()` falls back to `satorbit_rkf78m()` for an unknown method value

---

9. Configuration Reference

9.1 Force Model Options

The switches below belong to `force_model_opt_t`, one instance per satellite, owned by its `force_model_t`
(see 7.4).

Field	Values	Description
`gravity`	0 / 1	Earth gravity field switch
`shmopt`	`SHADOW_CYLINDRICAL` / `SHADOW_GEOMETRICAL` / `SHADOW_CONICAL`	Shadow model
`atmdrag`	0 / 1	Atmospheric drag switch
`atmdensopt`	`ATMODENSITY_HPM` / `ATMODENSITY_NRL`	Harris-Priester / NRLMSISE-00
`solrad`	0 / 1	Solar radiation pressure switch
`solradmdl`	`SOLRAD_MODEL_DEFAULT` / `ECOM1_5` / `ECOM1_9` / `ECOM2` / `ECOMC` / `CODE_5` / `CODE_9`	SRP model
`nbody`	0 / 1	Third-body attraction switch
`relativity`	0 / 1	Relativity switch
`solidtide` / `oceantide` / `poletide`	0 / 1, `OCEANTIDECORR_DEFAULT` / `_OTMODEL`	Tide corrections
`anelasticearth` / `rotcorr`	0 / 1	Anelastic earth, rotational deformation
`tidesystem`	`GRAVITY_MODEL_TIDE_ZERO` / `_TIDE_FREE`	Tide system of the gravity field
`gN`, `gM`	integers	Gravity field degree and order (example: 10 × 10)
`oL`, `oM`	integers	Ocean tide expansion degree and order (example: 10 × 10)
`udtransmatrix`	`UDTRANSMATRIX_INTGRTN` / `NUMDIF_2P` / `NUMDIF_3P`	Transition-matrix strategy
`udacclgrad`	`ACCLGRAD_ANALYSIS` / `NUMDIFF_ALL` / `NUMDIFF_SEP`	Acceleration-partial strategy
`usesamegravitymdl` / `_eop` / `_spw` / `_otm`	0 / 1	Share model data across satellites
`solradparas[8]`	`{area, mass, CR, P0, Au, …}`	SRP geometry and constants

Field	Values	Description
`atmdrgparas[8]`	`{area, mass, CD, …}`	Drag geometry and coefficient
`CR`, `P0`, `Au`, `D0` … `Ds4`	doubles	Per-satellite SRP parameters
`gfcfile` / `eopfile` / `spwfile` / `octfile` / `jplde440file`	paths	Model files

9.2 Processing Options

Everything below lives in `pod_opt_t`; the `estopt.*` rows are the nested `prcopt_t` read by the estimation
drivers.

Field	Description
`estmode`	Estimation mode, see 8.3
`udorbitint`	Integrator selection, see 9.3
`satvar[GNX]`, `satprn[GNX]`	Satellite state initial variance and process noise, per component
`rcvvar[GNRX]`	Receiver state process noise, per component
`satclkmode`	Satellite clock handling mode
`onlyusecode`	Use pseudorange only
`outldetexcs`	Enable MAD outlier detection and exclusion in the filter
`clkref`	Reference-clock station name
`navfile`, `navfile2`, `satantp`, `rcvantp`, `dcb`, `blq`, `snxfile`, `staposfile`	A priori data files
`obsbinfile`, `stabinfile`	Binary observation / station files for `podfbcmb()`
`estopt.mode`	RTKLIB positioning mode (`PMODE_*`)
`estopt.nf`	Number of frequencies; `2` gives the ionosphere-free combination
`estopt.navsys`	System bit mask actually processed
`estopt.elmin`	Elevation cutoff (rad)
`estopt.sateph`	A priori satellite ephemeris source (`EPHOPT_*`)

Field	Description
`estopt.ionoopt` / `tropopt`	Ionosphere and troposphere modelling (`IONOOPT_IFLC`, `TROPOPT_EST`, …)
`estopt.tidecorr`	`1` = solid earth tide only, otherwise 7 = solid + ocean loading + pole tide
`estopt.posopt[6]`	Correction switches, see the table below
`estopt.maxout`, `maxinno`, `thresslip`	Outage counter, innovation threshold, slip threshold
`estopt.eratio[]`, `err[]`, `std[]`, `prn[]`	Measurement and process-noise factors

`posopt[]` as actually used in `src/` (verified at `gpod.c:1601`, `1602`, `1605`, `1220`, `1757`):

Index	Effect	Implementation
0	Apply satellite antenna phase center/variation	`satantpcv()`
1	Apply receiver antenna phase center/variation	`antmodel()`
2	Apply phase wind-up	`model_phw()`
3	Exclude measurements of eclipsing block IIA satellites	`testeclipse()`
4	Set in the example, but **not referenced anywhere in `src/`** — currently without effect	—
5	Handle the day-boundary clock jump (reset ambiguities)	`udbias_pod()`

Defaults inherited from `prcopt_default_ppp` (`rtkcmn.c`): `maxout` = 5 epochs, `thresslip` = 0.05 m (geometry-free jump), `maxinno` = 30 m — the last one is declared but currently **not read anywhere in `src/`**, in the same spirit as `posopt[4]` above. The quality-control flow that consumes the first two is described in 4.8.

9.3 Integrator Selection

`udorbitint`	Value	Method	Backend	Dispatched to
`ORBITINT_GSLODE_RK8PD`	1	Runge-Kutta 8(7) Prince-Dormand	GSL `odeiv2`	`satorbit_gslode()`
`ORBITINT_GSLODE_RKF45`	2	Runge-Kutta-Fehlberg 4(5)	GSL `odeiv2`	`satorbit_gslode()`

`udorbitint`	Value	Method	Backend	Dispatched to
`ORBITINT_GSLODE_MSADAMS`	3	Adams implicit, variable order	GSL `odeiv2`	`satorbit_gslode()`
`ORBITINT_GSLODE_MSBDF`	4	BDF implicit, variable order	GSL `odeiv2`	`satorbit_gslode()`
`ORBITINT_RKF78M`	5	Runge-Kutta-Fehlberg 7(8)	built-in `rkf78m()`	`satorbit_rkf78m()`
`ORBITINT_ODEINTV2_ADAMSADAMS`	6	Adaptive Adams-Bashforth-Moulton	Boost.Odeint (case 0)	`satorbit_odeint()`
`ORBITINT_ODEINTV2_CONMSADAMS`	7	Controlled Adams-Bashforth-Moulton	Boost.Odeint (case 1)	`satorbit_odeint()`
`ORBITINT_ODEINTV2_RKF78`	8	Runge-Kutta-Fehlberg 7(8)	Boost.Odeint (case 2)	`satorbit_odeint()` — **used by the example**
`ORBITINT_ODEINTV2_RK_CASH_KARP54`	9	Cash-Karp 5(4)	Boost.Odeint (case 3)	`satorbit_odeint()`

9.4 Compile-Time Switches

These macros change the build configuration and must be set as compiler definitions (`-DNAME=value` or `add_compile_definitions`), since none of them is defined in the repository:

Macro	Default	Effect
`ENAGLO`	**not defined**	Enables GLONASS: sets `NSATGLO = 27` and `NSYSGLO = 1`
`ENAGAL`	**not defined**	Enables Galileo: `NSATGAL = 36`
`ENAQZS`	**not defined**	Enables QZSS: `NSATQZS = 10`
`ENACMP`	**not defined**	Enables BeiDou: `NSATCMP = 63`
`ENAIRN`	**not defined**	Enables IRNSS: `NSATIRN = 14`
`ENALEO`	**not defined**	Enables LEO satellites and the SBAS PRN range 120–158

Macro	Default	Effect
`OUTPUT_LOG`	`1` in `log.h`	Set to 0 to compile all logging to empty stubs
`IERS_MODEL`	**not defined**	Use `dehanttideinel_` instead of the built-in `tide_solid()`
`NFREQ`	3	Number of carrier frequencies
`NEXOBS`	3	Number of extended observation codes
`MAXOBS`	64	Maximum observations per station per epoch
`MAXSTRPATH`	1024	File-path buffer length
`GNRCVS` (in `podlib.h`)	135	Maximum number of receivers
`NSATGPS`	32	GPS satellite count, derived from `MINPRNGPS` / `MAXPRNGPS`

Important: because `MAXSAT` is assembled from those same macros, the **default build is GPS-only with `MAXSAT = 32`**. Enabling BDS, Galileo or GLONASS requires adding the corresponding `-DENA*` definitions *and* rebuilding the library and the application together — `MAXSAT` changes the size of `podobs_t`, of `pod_t::orbits[]` and of the estimated state, so mixed builds are not binary compatible.

9.5 State Process Noise in the Example

`test_podflt.c` configures the satellite state noise as follows (variance → standard deviation):

Component	`satvar[]`	1σ	`satprn[]`
Position	`SQR(5.0)`	5 m	`1E-30`
Velocity	`SQR(0.0001)`	0.1 mm/s	`1E-15`
SRP parameters	`SQR(0.1)`	0.1	`1E-15`
Clock	`SQR(3.0)`	3 m	`1.0`
Clock drift	`SQR(0.0001)`	0.1 mm/s	`1E-8`
Receiver position	`SQR(1E-3)`	1 mm	—
Receiver clock	`SQR(60.0)`	60 m	—
Receiver clock drift	`SQR(1.0)`	1 m/s	—
ZTD	`SQR(0.5)`	0.5 m	—

---

10. Data Preparation

10.1 Required Files

The program reads everything through **relative paths**; the following must exist in the working directory (these are the exact references in `test_podflt.c`):

Path	Purpose
`data/model/EOP-All.txt`	Earth orientation parameters (polar motion, UT1-UTC, LOD)
`data/model/SW-All.txt`	Space weather (F10.7 / Ap / Kp) for NRLMSISE-00
`data/model/GGM03C.gfc`	Gravity field coefficients
`data/model/de440coeff`	JPL DE440 planetary ephemeris coefficients
`data/model/fes2004_Cnm-Snm.dat`	Ocean tide potential coefficients (shipped under `lib/oceantide`)
`data/model/igs20p21130.snx`	IGS station coordinates (SINEX)
`data/model/igs_absolute_14.atx`	Satellite / receiver antenna phase centers
`data/model/P1C12004.DCB`	Satellite and receiver code biases
`data/model/FES2014b.BLQ`	Ocean loading displacements
`data/nav/BRDC00IGS_R_2020187*.rnx`	Broadcast ephemeris (RINEX 3)
`data/nav/igs2113*.sp3`	Precise ephemeris (SP3) used as the reference in `pode.sol`
`data/obs/podobs_2020187.bin`	Binary observation packets (a stream of `podobs_t`)
`data/obs/podsta_2020187.bin`	Binary station table (a stream of `sta_t`)

`EOP-All.txt` and `SW-All.txt` are shipped under `lib/nrlmsise/` and can be copied directly. The remaining model files must be obtained from the official sources (IGS, ICGEM, JPL, CODE).

10.2 File Format / Reader Map

File family	Format	Reader in this project
Observation	RINEX 2/3 (`.rnx`, `.obs`, `.crx`)	`readrnx()`, `readrnxt()`, `readpodobs_rnx()`
Navigation	RINEX 2/3 (`.rnx`, `.nav`, `.gnav`, `.hnav`, `.cnav`)	`readrnx()` + `uniqnav()`
Clock	RINEX clock (`.clk`)	`readrnxc()`
Precise orbit	SP3 (`.sp3`)	`readsp3()`, `readpodobs_sp3()`

File family	Format	Reader in this project
Satellite antenna	ATX	`readpcv()` / `readantex()` chain, `satantoff()`
Code bias	DCB (CODE `.DCB`, `.bsx`)	`readdcb()`
Station coordinates	SINEX (`.snx`)	`podudrcvsnx()` → `readsnx()`
Ocean loading	BLQ	`readblq()`
Gravity field	ICGEM `.gfc`	`readgfc()`
Earth orientation	IERS/Celestrak text	`readeop()`, `geteop()`, `geterp()`
Space weather	`SW-All.txt` style	`readspw()`
Ocean tide coefficient	FES2004 text	`readotfile()`
Planetary ephemeris	JPL DE440 binary coefficient file	`readjplde440coeffs()`
POD observation container	Binary `podobs_t` / `sta_t` dump	raw `fread()` in the application

10.3 Memory Footprint and Sizing

With the default build (`MAXSAT = 32`, `GNRCVS = 135`, `GNX = 23`, `GNRX = 13`, `GNDCB = 1`):

$$\begin{aligned} n_x &= \mathrm{MAXSAT} \cdot \mathrm{GNX} + \mathrm{GNRCVS} \cdot \mathrm{GNRX} + \mathrm{GNRCVS} \cdot \mathrm{MAXSAT} + \mathrm{GNDCB} \\ &= 32 \cdot 23 + 135 \cdot 13 + 135 \cdot 32 + 1 = 6812 \end{aligned}$$

Estimated state and memory footprint

default build: MAXSAT = 32, GNRCVS = 135, GNX = 23, GNRX = 13, GNDCB = 1



P and Pp are dense nx x nx doubles. H and R are dimensioned by the station count and the observations per epoch and are rebuilt (and freed) at every epoch - see section 10.3 for the sizing levers.

Allocation	Size
`x`, `xp`	$6812 \times 8\ \mathrm{B} \approx 54$ kB each
`P`, `Pp` (dense)	$6812^2 \times 8\ \mathrm{B} \approx 354$ MiB each, ≈ **708 MiB** together
Design matrix `H` per epoch (mode B, dense)	$n_v \times n_x \times 8\ \mathrm{B}$
Weight matrix `R` per epoch (mode B, dense)	$n_v \times n_v \times 8\ \mathrm{B}$

For example with 124 stations and 12 observations per station per epoch, $n_v = 124 \cdot 12 \cdot 2 + 32 \cdot 6 = 3168$, so `H` ≈ 165 MiB and `R` ≈ 77 MiB, allocated and freed every epoch. Levers, in order of effect:

1. reduce `GNRCVS` to the number of stations actually used (`podlib.h`),
2. process a subset of stations per epoch,
3. reduce `MAXSAT` if the constellation set allows it.

The 100 MB thread stack configured in `CMakeLists.txt` (`-Wl,--stack,104857600`) exists because the filter and force routines use several large stack arrays (for example the $3\times$ `GNX` partial buffers and the 23×23 blocks in `orbitdeq()`).

10.4 Test Dataset

Test data (125 global IGS stations) is available on Baidu Netdisk:

- Link: https://pan.baidu.com/s/15ZHs17U9rzWYmUXBbEvZwQ
- Access code: `ywaw`

---

11. Build

11.1 Requirements

Item	Requirement
Build system	CMake ≥ 3.16
Compiler	MinGW-w64 GCC or MSVC, C99 or later. A **Fortran compiler is also required**, because OpenBLAS/LAPACK is Fortran
Reference environment	Windows 10 + Intel Core i5-12500H (2.50 GHz) + CLion 2025.1.1 + MinGW-64
Link options	Windows links `Winmm`; a custom **100 MB thread stack**; Release uses `-O2`
Disk / time	`lib/` contains roughly 45 000 files; the first full build compiles OpenBLAS, GSL and Boost from source and takes a while

11.2 Build Steps

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . -j8        # Windows PowerShell: cmake --build . --config Release
```

Build target	Sources	Links against
`sofa`, `nrlmsise`, `openblas`, `gsl`, `odeint`	bundled source trees under `lib/`	`m`, plus the Fortran runtime (`gfortran`, `quadmath`) for OpenBLAS/LAPACK
`gpod` (static library)	`aux_source_directory(src ...)`: every file in `src/`	the five bundled libraries, plus `m` and `pthread` (`Winmm` on Windows)
`test_podflt` (executable)	`test_podflt.c`	`gpod`

The `gpod` target is a static library built from `aux_source_directory(src ...)`, so any new `.c` file placed in `src/` is compiled automatically.

11.3 Linking From Your Own Project

```
add_subdirectory(path/to/gpod)
target_link_libraries(your_app PRIVATE gpod)

# Required include directories:
```

```
#   gpod/src            (podlib.h, rtklib.h, log.h)
#   gpod/lib/sofa/src, gpod/lib/nrlmsise, gpod/lib
```

Because `podlib.h` includes `sofa.h`, `rtklib.h` and `log.h`, consumers only need the include directories above plus the same macros that were used to build the library.

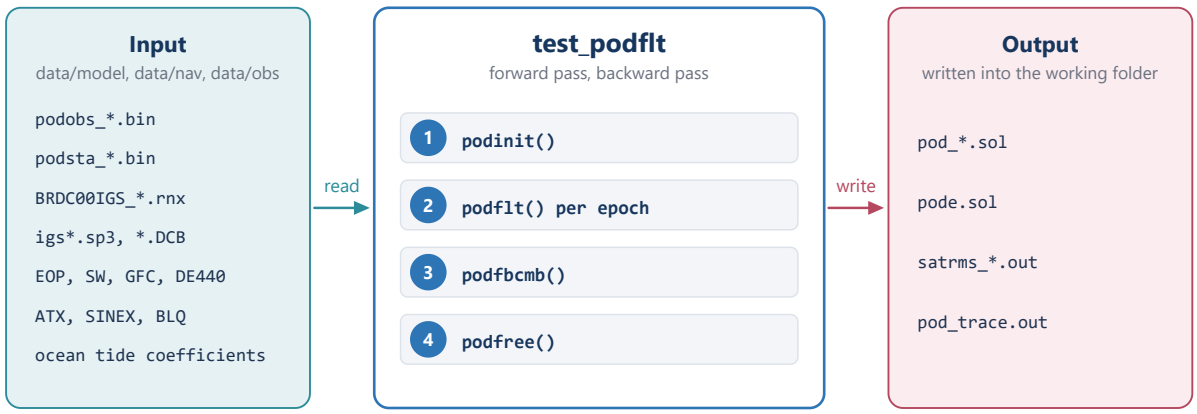
11.4 Troubleshooting

Symptom	Likely cause / remedy
`undefined reference to gfortran_*`, `quadmath_*`	Fortran runtime missing; install the MinGW-w64 Fortran component or link the Fortran runtime explicitly
`cannot find -lopenblas` / `-lgsl`	The bundled dependency libraries were not built; build the `openblas` / `gsl` targets first, or re-run the configure step
Stack overflow at run time	Increase `CMAKE_EXE_LINKER_FLAGS` stack size (already 100 MB by default on MinGW) or reduce `GNRCVS` / `MAXSAT`
`no enough memory` printed and `exit(0)`	`podinit()` failed to allocate `x` / `P` / `xp` / `Pp`; reduce `GNRCVS` / `MAXSAT` (see 10.3)
All logging silent	`OUTPUT_LOG` is 0, or `log_set_level()` was not called, or the trace level is lower than the message level
Solution is empty / all zeros	Model files failed to load (`fopen` failures are silent), or `clkref` does not match any station name
Satellites other than GPS never appear	The corresponding `ENA*` macro is not defined (see 9.4)

12. Running and Output

From input files to output files

One run consumes the three data folders and produces the result files



The driver filters the arc forward, replays it backward and combines the two solutions into pode.sol.

```
# run from a working directory that contains data/
./test_podflt
```

Workflow of the example:

1. Reads **124** stations sequentially from `podsta_*.bin` (`nstation = 124`; adjust to the actual record count — the file is not validated);
2. Configures the force model (10×10 gravity field and ocean tide, cylindrical shadow, NRLMSISE-00 density, ECOMC SRP, variational-equation transition matrix, analytic acceleration partials) and the estimation options (GPS, ionosphere-free combination, estimated troposphere,
   10° cutoff);
3. Calls `podinit()`, reads the RINEX navigation file and three days of SP3, opens the log;
4. **Forward filter**: calls `podflt()` for every `podobs_t` packet, writing `pode.sol` epoch by epoch;
5. **Backward filter**: rewinds the file packet by packet with `fseeko64` and filters the same observations in reverse time order;
6. Projects the residual $\Delta\mathbf{r} = \mathbf{x}_s^{\text{filter}} - \mathbf{r}_s^{\text{SP3}}$
   into the satellite body frame with `ecsf2satf()` and accumulates per-satellite statistics;
7. Prints `RMS[forward]` / `RMS[backword]` per satellite to stderr and dumps the residual series to file.

Output files:

File	Content
`pode.sol`	Per-epoch solution. Each epoch starts with `time=...` (GPST, 3 decimals). Per satellite: `PRN [DOP, #obs]`, then the position residual and its σ for X, Y, Z, then the velocity residual and σ (velocity is derived from a 1 ms finite difference of the SP3 reference), then clock residual, its σ, and the clock-drift residual, followed by the SRP parameters
`pod_trace.out`	Leveled debug log (level set by `log_set_level()`; 4 dumps matrices, 5 is verbose)
`satrms_forward.out`	Forward-filter satellite-frame residual series: three numbers per satellite per epoch, i.e. $3 \times \mathrm{MAXSAT}$ columns per line
`satrms_backword.out`	Same for the backward filter (filename spelling follows the source)
stderr	Progress (`POD processing: <time> nrcv=<n>`) plus per-satellite `RMS[forward]` / `RMS[backword]` in X, Y, Z and the 3D combined RMS

Residual buffers in the example are static and hold at most 1024 epochs per satellite (`drtns[3][MAXSAT][3][1024]`); longer arcs must increase that dimension or stream the residuals to file instead of buffering them.

---

13. Validation

The example is itself a validation harness: the filter solution is differenced against SP3 and the residuals are decomposed in the satellite body frame, which separates radial-like from along-track/cross-track behaviour and makes SRP modelling errors visible.

Step	Implementation
Reference position and velocity	`peph2pos(t, sat, &nav, 0, rs, dts, &var)` — SP3 position, velocity by 1 ms finite difference; `opt = 0` means the satellite antenna offset is **not** applied
ECEF → ECI reference	`satposeci()`
Relativistic clock term	`relcorr()` applied to the reference satellite clock
Difference	$\Delta\mathbf{r} = \mathbf{x}_s^{\text{filter}} - \mathbf{r}_s^{\text{ECI}}$
Frame rotation	`ecsf2satf(rs, E)` then `matmul("NN",3,1,3,1.0,E,dxyz,0.0,drtn)`
Statistics	`rms()` per axis plus the 3D combination, printed per satellite

Practical checks when you extend the code:

- The **filter-to-SP3 agreement is not an independent truth**: SP3 is also used inside the filter as an a priori/constraint source. For an independent assessment, re-run with SP3 removed from the filter and compare against a later IGS final product.
- Watch the **3D RMS per satellite** rather than per-axis values; a systematic offset in one axis usually points at attitude/SRP modelling (yaw mode or ECOM parameterisation).
- Compare forward and backward statistics: a large forward/backward asymmetry early or late in the
  arc is normal (edge effects), a growing difference in the middle indicates a filter or model problem.

---

14. Known Limitations

1. **Default build is GPS-only.** `MAXSAT = 32` unless `ENAGLO`, `ENAGAL`, `ENACMP`, `ENAQZS`, `ENAIRN` or `ENALEO` are defined; those macros are never defined in the repository (see 9.4). The satellite metadata table, RINEX code and force model
   are constellation-agnostic, but the compiled state layout is not.
2. **Windows-style hard-coded data paths.** `test_podflt.c` uses backslash paths such as `data\model\...`; on Linux/macOS these must be changed or moved into an external configuration.
3. **Binary observation packets are platform-bound.** `podobs_*.bin` / `podsta_*.bin` are raw memory

dumps of the `podobs_t` / `sta_t` structs (`fread(..., sizeof(podobs_t), 1, ...)`), so the layout
depends on compiler, alignment, macro settings (`MAXSAT`, `MAXOBS`, `NFREQ`) and struct version; files are **not portable across builds**.

4. **Hard-coded station count and reference clock.** The example uses `int nstation = 124;` (one less
than the "125 stations" mentioned in the data description) and hard-codes the fourth station (`stas[3].name`) as reference clock, without any validity check.
5. **Dense allocations.** `P` / `Pp` are `nx × nx` and each epoch allocates dense `H` and `R` (see 10.3); this scales poorly with `GNRCVS` and with the
number of observations per epoch.
6. **Example buffers are fixed size.** The residual arrays hold 1024 epochs per satellite; longer arcs overflow them.
7. **Mode handling in the example.** The output branch is chosen by `pod.opt.estmode`, while the actual filter branch is chosen by `obs.type` inside each binary packet; both must be consistent, and the example does not verify this.
8. **No input validation.** `fopen` return values are not checked in the example, configuration strings are copied with `strcpy` into `MAXSTRPATH = 1024` buffers, and SPD/stack limits are assumed to be sufficient.
9. **SRP and drag masses diverge.** `podinit()` sets the SRP mass from `gsatinfos[]` but leaves the
drag mass at `atmdrgparas[1]` (8000 kg in the example); per-satellite drag masses must be set in
`fmdlopts[sat]` explicitly.
10. **`posopt[4]` is set but unused.** The example enables it, but no code in `src/` reads it.
11. **`log.c` doc block is narrower than reality.** The comment documents levels 1–3, while the code
and `gint.c` also use levels 4 (matrix dumps) and 5 (verbose).
12. **Precise-version source.** For the professional version of the source code, contact the author.

---

15. Glossary

Term	Meaning
POD	Precise Orbit Determination
ECI / ECEF / ECSF / SATF	Earth-Centred Inertial, Earth-Centred Earth-Fixed, Earth-Centred Space-Fixed, satellite body-fixed frames
EOP / ERP	Earth Orientation Parameters: polar motion `xp`, `yp`, UT1-UTC, LOD, TAI-UTC
LOD	Length of Day

Term	Meaning
SRP	Solar Radiation Pressure
ECOM / CODE	Empirical solar-radiation-pressure parameterisations (D/Y/B frame based)
DYB frame	Satellite frame spanned by the Sun direction D, the solar-panel axis Y and B = D × Y
ZTD	Zenith Total Delay (troposphere)
IF combination	Ionosphere-Free linear combination of two frequencies
DCB	Differential Code Bias
PCV / PCO	Phase Center Variation / Offset (antenna)
ATX / BLQ / SNX / GFC / SP3 / SINEX	Antenna exchange, ocean loading, station coordinate, gravity coefficient, precise orbit, station solution file formats
EKF	Extended Kalman Filter
RTS smoother	Rauch-Tung-Striebel smoother
MAD	Median Absolute Deviation (robust outlier test)
LLI / GF / MW	Carrier-phase loss-of-lock indicator, geometry-free combination, Melbourne-Wübbena combination (cycle-slip tests)
DE440 / SOFA / GSL / OpenBLAS / NRLMSISE-00	JPL planetary ephemeris; IAU astronomy library; GNU Scientific Library; BLAS/LAPACK; empirical atmosphere model

---

16. References

1. RTKLIB: An Open Source Program Package for GNSS Positioning — http://www.rtklib.com
2. RTCM Standards — http://www.rtcm.org
3. RTCM NTRIP — http://software.rtcm-ntrip.org
4. IERS Conventions (2010), IERS Technical Note No. 36
5. IAU SOFA — Standards of Fundamental Astronomy, Release 2023-10-11
6. J. M. Picone et al., NRLMSISE-00 empirical model of the atmosphere, JGR, 2002
7. D. A. Vallado, *Fundamentals of Astrodynamics and Applications*, 4th ed.
8. GGM03C gravity model — ICGEM
9. JPL DE440 planetary ephemeris
10. `gpod.pdf` in this repository: derivation of the state equations, variational equations, observation model and filter formulation. The same material in English is in `gpod-en.md` and `gpod-en.pdf`, using the `gpod.pdf` equation numbers (1) to (23); section 4.7 below maps them to the numbering used here

17. Contact and Licensing

- Author: **Su Jinglan**
- QQ: `1971129844`
- Email: `1971129844@qq.com`

Note: if you need the source code of the **professional version**, please contact the author.

Source files carry the notice `Copyright(c) 2023-2025 by sujinglan, all rights reserved`. `src/rinex.c`, `src/ephemeris.c` and related files originate from RTKLIB (`Copyright (C) 2007-2020 by T.TAKASU`). `lib/sofa` is distributed under the IAU SOFA licence, `lib/gsl` under the GPL, and `lib/boost` under the Boost Software License — please verify each dependency's terms before use.